

# 第五课：融合实战——智能数据分析助手

## (LangChain + Function Calling + OpenClaw)

### 01

#### 课程主题

| 项目   | 内容                                                                                                                                         |
|------|--------------------------------------------------------------------------------------------------------------------------------------------|
| 课程主题 | 融合实战：智能数据分析助手——LangChain、Function Calling、Skills、OpenClaw                                                                                  |
| 课时安排 | 第3周第1次课（2小时）                                                                                                                               |
| 学员基础 | 已完成第1-4课，掌握Agent基础、Function Calling原理、ReAct等认知框架、LangChain基本使用                                                                             |
| 教学目标 | 1. 综合运用前四课所学技术解决真实业务问题<br>2. 深入理解 LangChain 和 OpenClaw 两种技术栈的架构设计与实现<br>3. 掌握数据分析类Agent的工具定义、记忆集成、多步推理等关键技术<br>4. 建立技术选型思维，能够根据实际场景选择合适的框架 |

### 02

#### 环境准备

- Python 3.11+ 环境
- 大模型 API 密钥（OpenAI / 国产模型）
- 预先安装：pandas、matplotlib、seaborn、langchain、langchain-openai、openclaw
- 预习：pandas基础操作（groupby、sort\_values、filter）

## 课程详情

### 第一部分：开场与上节回顾

#### 1.1 【开场问候】

同学们好，欢迎回到 AI Agent 全栈开发课程的第七课。

前几节课，我们完成了从零到一的系统学习。如果把 Agent 开发比作练武功，那前几节课我们就是在扎马步、练拳法、修内功。今天，我们要把这些招式全部组合起来，打一套完整的‘实战套路’组合拳。

#### 1.2 【前四课核心回顾】

老习惯，我们还是先来快速回顾一下前几节课的关键内容：

第一：我们介绍了 Agent 的认知革命——我们回答了‘Agent 到底是什么，为什么会产生 Agent’这两个根本问题。同时也深入理解了 Agent 的底层公式、技术演进路线，还搭建了 OpenClaw 环境。这是我们这套 Agent 智能体课程的认知基础。

第二：我们动手实战第一个 Agent。我们用 LangChain 搭建了带工具和记忆的 Agent，体验了 OpenClaw 的快速上手。这是从理论到实践的跨越。

第三：深度解析 Function Calling——我们深入了 Agent 能够‘动手’的底层原理和技术基石。我们从底层机制到全链路解析进行深入剖析，甚至用原生 SDK 模拟了两次 API 调用。这是技术深度的体现。

第四：理解 Agent 认知框架——我们学习了四种思维模式：ReAct（动态决策）模式、Plan-and-Execute（高效执行）模式、Self-Ask（知识检索）模式、Reflection（自我反思）模式。这是 Agent 的‘大脑’如何关注思考的问题。

学到这里，相信大家已经掌握了 Agent 开发的全部‘零件’。

今天，我们要再次把这些零件组装起来，造一台真正能用的机器。

#### 1.3 【引出本节课主题】

本节课是一堂动手实操课，目的是解决一个真实的业务问题，那就是实现一个“智能数据分析助手”。

想象一下：你是一家电商公司的首席数据运营官。每天早上，老板都会在群里@你：

- ‘昨天哪个产品卖得最好？’
- ‘华东区的利润为什么下降了？’
- ‘给我画一张月度销售趋势图，放到周报里。’

这就是你每天要面对的事情，平日里你的处理应对方式就是打开 Excel 文件。如果你会编码，你就会手写几行 pandas 代码；如果你是 Office 小王子或者小仙女，你可能会在 Excel 中拉个透视表，然后截图、复制、粘贴、发送……每天重复，耗时又无聊。

今天，我们就打造一个智能助手智能体 Agent 雏形。只需要说一句话，它就能：

- 第一：自动理解你的意图
- 第二：调用数据分析工具
- 第三：生成可视化图表
- 第四：返回简洁的答案

我们只需要动嘴，它就会帮我们干活。

我们会采用两种不同的技术栈来解决这个问题：

- **方案A：**我们会基于 LangChain 框架——手动定义工具函数（`@tool`），使用 `create_agent` + `StateGraph` 实现记忆和多步推理。这种场景是适合需要精细控制的严肃生产场景经常采用的解决方案。我们从工具定义到记忆集成，完整实现一个数据分析 Agent。
- **方案B：**我们会基于 OpenClaw Skills —— 将分析能力封装为技能，利用 OpenClaw 其内置的记忆和多模型路由来完成。这种方案特别适合快速开发和生态共享的工作场景。将分析能力封装成技能，让大家体验配置化开发的效率。

通过两种方案的技术设计，我们也会从多个维度对比两种方案，给出决策树和混合使用策略。

相信通过今天课程的两种技术方案对比，大家将深入理解两种技术栈的设计哲学和适用场景，建立起技术选型的思辨能力。

## 1.4 【过渡：进入场景分析】

好了，目标清楚了。接下来，我们先进入第一部分——场景分析与架构设计。

在动手写代码之前，我们必须要做一件事：**把上述电商老板的‘一句话需求’拆解成技术可执行的能力清单。**

这是我们从‘会写Agent’到‘会设计Agent’的关键一步，这也是以后一人公司或者超级个体必须首先要具备的能力，从用户的“一句话需求”到“需求分析，并进行具像化行动策略”的能力。

## 第二部分：场景分析与架构设计

### 2.1 业务场景与需求拆解

#### 2.1.1 【引入：从“模糊需求”到“清晰能力”】

假设我们平日经常操作的是一份电商销售数据的 Excel 文件，它的文件结构包含以下字段：

| 字段       | 说明   | 示例             |
|----------|------|----------------|
| month    | 月份   | Jan, Feb       |
| region   | 地区   | East, West, So |
| product  | 产品名称 | 产品A, 产品        |
| sales    | 销售额  | 1000, 1500     |
| profit   | 利润   | 200, 300       |
| quantity | 销量   | 100, 150       |

- Month —— 月份
- Region —— 地区
- Product —— 产品名称
- Sales —— 销售额
- Profit —— 利润
- Quantity —— 销量

#### 2.1.2 【三类典型问题】

日常工作场景，老板经常会问以下三类问题，每一类问题都会对 Agent 的能力要求有所不同。

##### 第一类问题：数据查询

比如：老板会关心公司‘销售额最高的月份是几个月？’ ‘华东地区的总利润是多少？’ ‘产品A在4月的销量是多少？’

这类问题只需要**精确的数据检索和计算**。这就要求我们的智能体 Agent 具备能从 CSV 中快速找到最大值、求和、筛选的能力。它既不需要画图，也不需要长篇分析，只要快速并准确地给出数字就行。

## 第二类问题：数据可视化

比如：销售总监关注经营数据变化趋势。她常会说‘画一张月度销售趋势图，或者是’ ‘按地区画出利润分布的柱状图。’ 等等。

这类问题我们经过分析，要求我们的智能体 Agent 不仅可以动态**生成图表，还要能调用 python `matplotlib` 库**，甚至还要知道横轴是什么、纵轴是什么、标题怎么写等等。更重要的是，它还需要把图表文件保存下来，让用户可以直接放进周报。

## 第三类问题：多步分析

比如：‘分析一下为什么华东区的利润下降了。’

这类问题是最难的一类，它要求更高的智能。我们拆解一下这类 Agent 的落地逻辑。Agent 首先需要先检索和分析数据（比如：获取华东区的利润变化趋势），然后分析原因（可能是销量下降、成本上升、或者产品结构变化等等），最后给出结论。这就要求 Agent 具备**多步推理和外部知识检索能力**。

好，经过分析，我们发现这三类问题的复杂度是依次递增的。而我们要发开的“智能助手”必须**全部覆盖**以上需求。

### 2.1.3 【五大核心能力】

下面先给出结论，那就是要解决以上这三类问题，我们的智能助手需要具备以下五大能力：

#### 第一大能力：自然语言理解能力

通常情况下，我们的用户并不会说‘对sales列执行max操作’，因为这不是他们的业务语言和交流叙事。

他们/她们经常说的是‘销售额最高的月份是几个月’。

这就要求智能体 Agent 要能理解‘最高’对应`max`，‘月份’对应`month`列。这不是简单的关键词匹配，而是**语义理解**。

#### 第二大能力：工具调用能力

我们要明白，智能体 Agent 自己并不会算数据，也不会画图。它需要调用外部工具——比如：用`pandas` python lib 库做数据操作，用`matplotlib` python 库进行画图，然后用`seaborn` 组件进行美化。这就要求 Agent 具备**Function Calling** 能力，能自主决定调用哪个工具、并抽取出针对不同工具需要填写什么参数。

### 第三大能力：记忆能力

用户的提问可能会连续的多轮对话，比如：‘总销售额多少啊？’ ‘那利润呢？’ ‘再画个图吧。’ 这就要求我们的智能体 Agent 需要能记住以前的话题，不需要用户重复提及。这就是需要支持**多轮对话记忆能力**。

### 第四大能力：多步推理能力

当我们对于‘为什么利润下降了’这类问题，Agent 往往并不能一步到位。它首先要自己分解步骤，比如：先查利润变化，再查销量变化，再查成本变化……最后才能综合得出结论。

这就要求我们的智能体 Agent 具备**规划能力**，这里我们会用到上节课学的 Plan-and-Execute 认知模式。

### 第五大能力：反思与修正能力

但假如碰到这样的结果，比如智能体 Agent 算出来的利润下降率是 200%，但财务老师经过核算却发现狂赚了几千万。这就是非常明显的异常。这就要求我们设计和开发的智能体 Agent 需要具备**自我检查和反省的能力**，它需要清楚是不是我们数据读错了？是不是计算公式用错了？并尝试修正。最终经过反思和更改后，把正确的答案返还给用户。

## 2.1.4 【小结与过渡】

好，简单总结一下上面的应用场景以及我们的分析：

1. 这是一个电商场景
2. 数据表有 6 个字段
3. 用户会问三类问题
4. 智能体 Agent 需具备五大核心能力——自然语言理解、工具调用、记忆、多步推理以及反思与修正。

有了这张‘能力清单’，接下来我们就可以进入到架构设计环节了。

## 2.2 技术架构设计

### 2.2.1 【开场：从“能力清单”到“架构蓝图”】

就像盖房子，能力清单是‘户型图’，技术架构是‘施工图’。

有了施工图，我们才能动手砌墙、布水电，这样我们才不至于行动走样，出现大的纰漏。

再声明一下，架构能力也是个人公司和超级个体 OPC 需要具备的核心能力之一。它考验的是我们的抽象能力。

下面，我们就来看下同一种生产场景的两种技术架构方案。

### 2.2.2 【方案A：LangChain架构图解读】

先看**方案A：LangChain 架构**。

这张简单的架构图分为三层：表现层、业务层、能力层。该架构图是程序员的纯技术视角。

**首先最上面的是表现层：**它可以是命令行或者 Web 界面。用户在这里输入问题，Agent 在这里返回答案。这一层我们通常用 `input()` 和 `print()` 搞定，或者用 FastAPI 包装成 WEB REST 服务形式。

**其次是中间层。该层是业务层 —— 我们使用 LangGraph 状态图来实现。**它是整个 Agent 的‘大脑’。在该层，我们设计三个节点：

- **第一个节点：规划节点。**它的作用是负责把用户问题拆解成步骤列表。比如‘分析利润下降原因’被拆成：查利润变化→查销量变化→查成本变化→综合分析。
- **执行节点：**按顺序执行每个步骤，调用下面的工具。
- **反思节点：**检查执行结果是否合理，如果不合理，触发修正或重规划。

这三个节点通过边连接，可以形成循环——执行完发现有问题，回到规划节点重新规划。这就是我们上节课学的 Plan-and-Execute + Reflection 模式的融合。

**最下面是能力层 —— create\_agent + @tool：**

- `create_agent` 方法创建 ReAct Agent，负责决策‘下一步该调用哪个工具’。
- `@tool` 装饰器把普通函数（如数据查询、绘图）包装成 Agent 可调用的工具。

**记忆怎么实现？**这里我们还是采用 LangGraph 的 `checkpointner`，后续大家也可以采用专门的开源记忆体来实现。当我们使用 `checkpointner`，每次当我们执行完一个节点，状态就会自动保存。下次

当用户再说话时，就会通过同一个 `thread_id` 加载之前该用户的所有信息上下文，从而实现多轮记忆。

上面这套架构的核心思想就是分层解耦，精细化控制。

实际开发你也可以充分发挥你的想象力，随意替换某一层，比如把 `create_agent` 换成自定义的 `StateGraph`，或者把 `checkpointer` 从内存换成数据库等等。

### 2.2.3 【方案B：OpenClaw架构图解读】

讲完 Langchain 方案后，下面我们再来看一下**方案B：OpenClaw 架构**。

这张图更简洁。最上层是 **OpenClaw 核心引擎**——它已经内置了 ReAct 循环、记忆管理、多模型路由，并通过合理有效的机制确保这些底层能力可以有效协同。我们不再需要自己画状态图。

中间层是**技能管理器**。每个能力都被封装成一个独立的技能，比如：

- `data_analyzer` 技能：负责数据查询
- `chart_generator` 技能：负责绘图
- `insight_engine` 技能：负责深度洞察分析

而且每个技能都由两部分所组成：

- `SKILL.md`：用自然语言描述技能的功能、参数、触发条件等。这部分技能描述是交给**模型**看的。
- `__init__.py`：具体 Python 代码实现，这是真正发挥技能，实际干活的代码。这部分是**机器**执行的。

最下面一层是**内置能力层**——它由记忆、多模型路由、多平台支持等底层能力所组成，开箱即用。你并不需要像 LangChain 一样编码配置 `checkpointer`，也不需要写 `thread_id`，OpenClaw 框架会自动处理和管理全流程。

**OpenClaw 这套架构的核心思想便是：配置优先，开箱即用。**你只需要关注‘技能要做什么’，其他剩下的可以完全交给框架发挥。



## 2.2.4 【核心差异：手搓 vs 配置】

现在，我们从架构组成可以清晰地看到两种方案的差异：

**LangChain 需要你‘手搓’所有细节。你需要：**

- 第一：手动定义状态图（规划节点、执行节点、反思节点）
- 第二：手动配置检查点（`checkpoint`），用来管理记忆
- 第三：手动管理 `thread_id`，保证个人会话跟踪
- 最后：手动处理工具调用的循环

这种方案的好处是**极度灵活，业务稳定可控**——你可以任意定制流程，比如加入人机协同、动态重规划、并行执行等等。坏处则是**代码量大**，一个简单的数据分析 Agent 可能要写几百行。

**而 OpenClaw 架构模式让你只关注‘配置’即可。你只需要：**

- 第一：利用自然语言写 `SKILL.md` 文档，描述具体技能
- 第二：用 Python 进行编码实现技能工具
- 第三：运行 OpenClaw 并添加 skills

OpenClaw 龙虾方案的好处是**开发效率高**——一个技能几十行代码就能跑起来。坏处是**灵活性有限，运行不可控**——如果 OpenClaw 内置的 ReAct 循环不满足你的需求，你很难自定义。

打个比方：LangChain 就是‘手动挡汽车’，你踩离合、换挡、加油门、甚至改装，全部必须自己控制，如果开得好能漂移，而且生死看淡，不服就干；但 OpenClaw 则像‘自动挡汽车’，你只管最基础的操作，如：踩油门和刹车，简单省心，但想漂移就比较难了，还是不够随心所欲。

## 2.2.5 【过渡：进入实现阶段】

好了，架构图讲完了。我们现在有了两张清晰的施工图。

接下来，我们要按照这两张图，分别动手实现。

先说**方案A的执行步骤**：因为我们是基于 **LangChain** 实现。我还是会带着大家从数据准备开始，接着定义三个核心工具（数据查询工具、绘图工具和分析工具），然后构建带记忆的 Agent，最后测试效果。

下面我们一一开始。

## 2.3 数据准备与工具预览

### 2.3.1 【开场：工欲善其事，必先利其器】

首先，我们需要一份数据。没有数据，数据分析 Agent 就是无米之炊。

我们准备一个简单的销售数据 CSV 文件，该文件包含三个月的销售记录。

这个数据虽然不大，但足够覆盖我们所需要的查询、可视化和分析三种场景。

### 2.3.2 【创建数据与代码演示】

大家打开终端，创建一个目录 `5-overall`，在该目录下创建一个 Python 文件（`create_data.py`），或者你也可以直接在 Jupyter Notebook 里编辑。

`create_data.py` 文件如下：

代码块

```
1  import pandas as pd
2
3  data = {
4      "month": ["Jan", "Jan", "Jan", "Feb", "Feb", "Feb", "Mar", "Mar", "Mar"],
5      "region": ["East", "West", "South", "East", "West", "South", "East",
6      "West", "South"],
7      "product": ["A", "B", "A", "B", "A", "B", "A", "B", "A"],
8      "sales": [1000, 1500, 1200, 1300, 1400, 1100, 2000, 1800, 1600],
9      "profit": [200, 300, 240, 260, 280, 220, 400, 360, 320],
10     "quantity": [100, 150, 120, 130, 140, 110, 200, 180, 160]
11 }
12 df = pd.DataFrame(data)
13 df.to_csv("sales.csv", index=False)
14 print(df)
```

下面我们简单解释下该代码作用：

代码块

```
1 import pandas as pd
```

首先我们导入 **pandas** 库。pandas 库是 Python 中用于数据分析和处理表格数据的核心库。

代码块

```
1 data = {
2     "month": ["Jan", "Jan", "Jan", "Feb", "Feb", "Feb", "Mar", "Mar", "Mar"],
3     "region": ["East", "West", "South", "East", "West", "South", "East",
4               "West", "South"],
5     "product": ["A", "B", "A", "B", "A", "B", "A", "B", "A"],
6     "sales": [1000, 1500, 1200, 1300, 1400, 1100, 2000, 1800, 1600],
7     "profit": [200, 300, 240, 260, 280, 220, 400, 360, 320],
8     "quantity": [100, 150, 120, 130, 140, 110, 200, 180, 160]
9 }
```

接着我们创建一个数据字典 **data**，该字典一共包含 6 个键值对，用来模拟销售数据：

1. **month**: 月份（这里只模拟1至3月）
2. **region**: 地区（这里指定东/西/南三个区域循环）
3. **product**: 产品（A/B两种产品交替）
4. **sales**: 销售额（为了简便我们一概采用整数，后续的利润和销量我们也是采用整数）
5. **profit**: 利润（整数）
6. **quantity**: 销量（整数）

统计下来一共 **9 行数据**，代表 9 条销售记录。

代码块

```
1 df = pd.DataFrame(data)
```

紧接着我们调用 **DataFrame** 方法，用来将字典数据类型转换为 **DataFrame** 数据类型。DataFrame 数据类型是 pandas 的核心数据结构，它类似 Excel 表格，有行有列。

代码块

```
1 df.to_csv("sales.csv", index=False)
```

DataFrame 数据结构转换完毕后，下一步我们就可以将 DataFrame 数据结构保存为 CSV 文件，这里 `index=False` 的含义是不保存行索引（0,1,2...）。

最后我们在控制台打印 DataFrame 结果，用来显示表格内容。输出结果如下：

代码块

|   | month | region | product | sales | profit | quantity |     |
|---|-------|--------|---------|-------|--------|----------|-----|
| 1 | 0     | Jan    | East    | A     | 1000   | 200      | 100 |
| 2 | 1     | Jan    | West    | B     | 1500   | 300      | 150 |
| 3 | 2     | Jan    | South   | A     | 1200   | 240      | 120 |
| 4 | 3     | Feb    | East    | B     | 1300   | 260      | 130 |
| 5 | 4     | Feb    | West    | A     | 1400   | 280      | 140 |
| 6 | 5     | Feb    | South   | B     | 1100   | 220      | 110 |
| 7 | 6     | Mar    | East    | A     | 2000   | 400      | 200 |
| 8 | 7     | Mar    | West    | B     | 1800   | 360      | 180 |
| 9 | 8     | Mar    | South   | A     | 1600   | 320      | 160 |

同时我们也会在当前目录生成 `sales.csv` 文件。

这个数据有6列，分别是月份、地区、产品、销售额、利润、销量。一共9行，代表3个月、3个地区、2种产品（A和B交替）的销售记录。

### 2.3.3 【数据结构解读】

数据生产完毕后，下一步我们再仔细看一下数据结构，因为它直接决定了我们的 Agent 能回答什么问题。

- **month**：Jan、Feb、Mar，三个月。我们可以按月做趋势分析。
- **region**：East、West、South，三个地区。可以按地区做对比。
- **product**：A和B两种产品。可以分析产品贡献。
- **sales**：销售额。用来回答 ‘卖了多少’ 。
- **profit**：利润。用来回答 ‘赚了多少’ 。
- **quantity**：销量。用来回答 ‘卖了多少件’ 。

注意观察，其实数据是有规律分布的：

- 1月：East卖A（1000） ， West卖B（1500） ， South卖A（1200）
- 2月：East卖B（1300） ， West卖A（1400） ， South卖B（1100）
- 3月：East卖A（2000） ， West卖B（1800） ， South卖A（1600）

这种设计可以让我们测试各种查询：比如‘产品A的销售额趋势’、‘East地区各月利润’、‘销量最高的产品’等等。

**为什么数据要这么设计呢？** 因为我们会在后面定义的 `query_sales_data` 工具需要支持多种查询模式。这个数据集虽然小，但包含了单条件查询、分组聚合、排序、筛选等典型操作。如果你能在这个数据集上把 Agent 调通，相信即使换成百万行数据也只需要改一下文件路径即可。

### 2.3.4 【预览即将实现的工具】

有了数据，接下来我们就要开始声明三个核心工具了。这三个工具分别是：

1. `query_sales_data`：数据查询工具。用户问‘总销售额多少’，它就用 pandas 算出来返回字符串。
2. `plot_sales_data`：绘图工具。用户说‘画一张趋势图’，它就调用 matplotlib 生成图片并保存。
3. `analyze_sales_trend`：分析工具。用户问‘为什么利润下降’，它就计算变化并给出初步洞察。

这三个工具分别对应我们之前说的那三类问题：数据查询、可视化、多步推理和分析。

因为我们是使用 LangChain 框架进行开发，所以这三个方法都会被 `@tool` 这样的装饰器进行包装，然后被传递给 `create_agent` 方法。智能体 Agent 会根据用户问题的语义，自动决定到底该调用哪个工具、以及具体填什么参数等等。

### 2.3.5 【过渡：进入工具定义】

好，现在数据准备好了，工具也简单预告了下。下面我们就正式进入**方案A：LangChain实现**的第一步——定义这三个工具函数。

## 第三部分：方案A——基于 LangChain 的实现

## 3.1 环境准备与工具定义

正式开发之前，我们还是先准备 LangChain 环境。下面我们进入 `5-overall` 项目目录，创建和激活虚拟环境，然后安装依赖，最后准备代码。

### 下面我们进入步骤1，初始化整个项目

代码块

```
1  mkdir 5-overall
2  cd 5-overall
3  python -m venv venv
4  source venv/bin/activate
5  pip install langchain langchain-openai langgraph pandas matplotlib python-dotenv
```

环境准备好后，下一步我们进入步骤2。

### 步骤2：需要导入依赖并加载数据

代码块

```
1  import os
2  import pandas as pd
3  import matplotlib.pyplot as plt
4  from dotenv import load_dotenv
5  from langchain_openai import ChatOpenAI
6  from langchain_core.tools import tool
7  from langchain.agents import create_agent
8  from langgraph.graph import StateGraph, END, add_messages
9  from langgraph.checkpoint.memory import InMemorySaver
10 from typing import TypedDict, Annotated, List, Tuple
11
12 load_dotenv()
13
14 # 加载数据（全局变量，实际应用可改为动态加载）
15 df = pd.read_csv("sales.csv")
```

简单解释下这些依赖，这些依赖是以后开发中你会不停地遭遇。学习知识到掌握知识是一个不停强化记忆的过程，学习和获取知识很容易，但是把知识内化为自身能力一般很难速成，需要坚持不懈的强化学习。知易行难，任重道远，大家一起共勉。好，下面我们打起精神，好好熟悉这些依赖。

代码块 `import os`

**os 是标准库**，用于给操作系统打交道用，比如读取环境变量、文件路径操作等等。

代码块

```
1 import pandas as pd
```

**pandas 库**，简称为 **pd**，是用于数据处理和分析，核心数据结构是 DataFrame（表格）。在 AI 世界里，模型少不了跟数据打交道，所以 pandas 是经常使用的数据处理核心库。

代码块

```
1 import matplotlib.pyplot as plt
```

**matplotlib 库**，主要用于绘图，简称为 **plt**。它的作用是用于数据可视化和绘制图表等等。

代码块

```
1 from dotenv import load_dotenv
```

**dotenv 库**是我们的老相识了，主要作用用于加载 **.env** 文件中的环境变量（比如 API 密钥等），这样可以避免硬编码敏感信息。

代码块

```
1 from langchain_openai import ChatOpenAI
```

**langchain\_openai 库**主要是从 LangChain 框架中调用 OpenAI 大模型（如 GPT-4）的封装类，用来跟大模型进行交互。

代码块

```
1 from langchain_core.tools import tool
```

**langchain\_core 库**用于导入 **tool** 装饰器。用于将普通 Python 函数标记为"工具"，将来供大模型调用。

代码块

```
1 from langchain.agents import create_agent
```

`langchain.agents` 库也是我们的老相识，我们经常会使用它的 `create_agent` 函数，作用是创建智能体（Agent），方便让大模型自主决策调用工具。

代码块

```
1 from langgraph.graph import StateGraph, END, add_messages
```

`langgraph.graph` 库 比较重要，这里会导入三个核心组件：

| 组件                        | 作用              |
|---------------------------|-----------------|
| <code>StateGraph</code>   | 状态图，用于构建工作流/流程图 |
| <code>END</code>          | 结束节点标记          |
| <code>add_messages</code> | 消息累加函数，用于对话历史管理 |

代码块

```
1 from langgraph.checkpoint.memory import InMemorySaver
```

紧接着 `langgraph.checkpoint.memory` 库我们也会经常看到，它的作用是导入内存检查点保存器。用于在 `langgraph` 工作流中保存状态，将来好实现多轮对话记忆功能。

代码块

```
1 from typing import TypedDict, Annotated, List, Tuple
```

再往下就是 `typing` 类型库，它会导入各种数据类型，比如这里的：

| 类型                       | 用途              |
|--------------------------|-----------------|
| <code>TypedDict</code>   | 定义带类型的字典（类似结构体） |
| <code>Annotated</code>   | 给类型添加元数据注释      |
| <code>List, Tuple</code> | 列表、元组数据类型类型     |



依赖导入工作结束后，下一步就是从当前目录的 .env 文件里加载环境变量

代码块

```
1 load_dotenv()
```

并且将 `KEY=value` 设置为系统环境变量，我们这里是读取大模型的 API KEY，将来好调用大模型用。

代码块

```
1 # 加载数据（全局变量，实际应用可改为动态加载）
2 df = pd.read_csv("sales.csv")
```

最后一步就是读取刚才我们创建的模拟市场数据 CSV 文件内容，并把这些数据从硬盘加载到内容，然后保存到 DataFrame 数据结构对象里。

简单总结来看，上面这段代码就是 AI 数据分析 Agent 的基础准备，它主要完成以下四件事：

1. 第一件事：数据工具准备（pandas + matplotlib），一个用来数据处理，一个用来数据可视化
2. 第二件事：准备大模型接入（LangChain + OpenAI）
3. 第三件事：初始化 Agent 框架。
4. 第四件事：数据预加载（sales.csv）

经过这一系列的操作之后，环境依赖就准备到位了。接下来我们就准备进入步骤三：定义工具函数、 workflow 节点，并构建一个可以自动分析数据，并生成图表的 AI Agent。

### 步骤3：定义数据分析工具

下面我们进入步骤3，定义数据分析工具阶段。下面我们依次会创建 3 个工具。

第一个工具：数据查询函数工具。它的作用是用 pandas 执行数据分析操作。

代码块

```
1 @tool
2 def query_sales_data(question: str) -> str:
3     """
4     对销售数据执行分析查询。支持以下类型的问题：
5     - 最高/最低值查询：如'销售额最高的月份'、'利润最低的地区'
6     - 总和/平均值：如'总销售额'、'平均利润'
7     - 分组统计：如'各地区的销售额'、'各月份的总利润'
8     - 条件筛选：如'产品A在3月的销量'
```

```

9
10     参数:
11         question: 用户的分析问题
12
13     返回:
14         分析结果的字符串描述
15     """
16     global df
17     q = question.lower()
18
19     # 最高值查询
20     if "最高" in q or "最大" in q:
21         if "销售额" in q:
22             max_row = df.loc[df['sales'].idxmax()]
23             return f"销售额最高的记录是{max_row['month']}月, {max_row['region']}地
24 区, {max_row['product']}产品, 销售额为{max_row['sales']}"
25         elif "利润" in q:
26             max_row = df.loc[df['profit'].idxmax()]
27             return f"利润最高的记录是{max_row['month']}月, {max_row['region']}地
28 区, 利润为{max_row['profit']}"
29
30     # 总和查询
31     if "总销售额" in q:
32         total = df['sales'].sum()
33         return f"总销售额为{total}"
34     if "总利润" in q:
35         total = df['profit'].sum()
36         return f"总利润为{total}"
37
38     # 分组统计
39     if "各地区" in q and "销售额" in q:
40         result = df.groupby('region')['sales'].sum().to_dict()
41         return f"各地区销售额: {result}"
42     if "各月份" in q and "利润" in q:
43         result = df.groupby('month')['profit'].sum().to_dict()
44         return f"各月份利润: {result}"
45
46     # 条件筛选
47     if "产品" in q and "销量" in q:
48         # 提取产品名 (简单处理)
49         for product in df['product'].unique():
50             if product in q:
51                 result = df[df['product'] == product]['quantity'].sum()
52                 return f"{product}产品总销量为{result}"
53
54     return f"无法理解问题: {question}。请尝试其他表述方式。"

```

下面我们简单过一下这个工具函数逻辑：

代码块

```
1 @tool
```

`@tool` 是 LangChain 装饰器，目的是将此函数注册为"工具"，方便大模型识别并调用。

代码块

```
1 def query_sales_data(question: str) -> str:
```

`query_sales_data` 函数中 `question` 表示用户自然语言问题，函数返回的结果是字符串数据类型。

**下面这段用三个双引号注释的信息是文档字符串，这段字符串是给大模型看的"说明书"。**大模型通过阅读这段描述，理解这个工具能做什么、以及参数怎么传。

代码块

```
1 """
2 对销售数据执行分析查询。支持以下类型的问题：
3 ...
4 """
```

这里的 `df` 变量使用的是外部定义，它的值是之前我们加载的 `sales.csv` 文件里的数据。

我这里只是为了方便采用了 `global` 关键词，大家在实际生产环境并不推荐用全局变量这样的骚操作，大家注意下。

这里的 `q` 是把用户的自然语言问题字符串转为小写，目标是方便后续关键字匹配时，可以不区分大小写。

代码块

```
1 q = question.lower()
```

**对 `q` 做条件判断**，检测用户问题中是否包含"最高"或"最大"这样的关键词。

代码块

```
1 # 最高值查询
2 if "最高" in q or "最大" in q:
```

紧接着下面这个条件语句作用是**查找销售额最高行**：

代码块

```
1         if "销售额" in q:
2             max_row = df.loc[df['sales'].idxmax()]
```

其中：

| 代码                                | 作用                   |
|-----------------------------------|----------------------|
| <code>df['sales'].idxmax()</code> | 找到 sales 列最大值的索引（行号） |
| <code>df.loc[...]</code>          | 根据索引获取整行数据           |
| <code>max_row</code>              | 包含该行所有列的字典/Series    |

接下来语句就是从 `max_row` 变量中取出 csv 文件中销售额最高行的各字段，然后拼接并进行格式化后返回。

代码块

```
1         return f"销售额最高的记录是{max_row['month']}月，{max_row['region']}地区，{max_row['product']}产品，销售额为{max_row['sales']}"
```

如果用户询问“总销售额”，则通过 `df['sales'].sum()` 函数对“sales”整列进行求和计算。

代码块

```
1         # 总和查询
2         if "总销售额" in q:
3             total = df['sales'].sum()
4             return f"总销售额为{total}"
```

如果用户希望查询各地区的销售额，则可以通过分组聚合函数进行汇总，如下所示：

代码块

```
1         # 分组统计
2         if "各地区" in q and "销售额" in q:
3             result = df.groupby('region')['sales'].sum().to_dict()
4             return f"各地区销售额：{result}"
```

| 代码                                | 作用                                                       |
|-----------------------------------|----------------------------------------------------------|
| <code>df.groupby('region')</code> | 按 region 列分组                                             |
| <code>['sales'].sum()</code>      | 每组内对 sales 求和                                            |
| <code>.to_dict()</code>           | 转为字典格式: <code>{ 'East': 4300, 'West': 4700, ... }</code> |

如果用户想知道某种产品的销量，则可以通过条件过滤和求和相结合的方式获取，如下代码所示：

代码块

```
1      # 条件筛选
2      if "产品" in q and "销量" in q:
3          for product in df['product'].unique():
4              if product in q:
5                  result = df[df['product'] == product]['quantity'].sum()
6                  return f"{product}产品总销量为{result}"
```

这里：

| 代码                                        | 作用             |
|-------------------------------------------|----------------|
| <code>df['product'].unique()</code>       | 获取所有产品名称（去重）   |
| <code>df[df['product'] == product]</code> | 筛选出该产品的所有行     |
| <code>['quantity'].sum()</code>           | 对 quantity 列求和 |

如果用户的问题都不在上面的判断条件内，也就是当所有条件都不匹配用户问题的话，则默认返回下列提示信息。

代码块

```
1      return f"无法理解问题：{question}。请尝试其他表述方式。"
```

至此，销售数据分析查询这个工具就介绍完毕了。这里需要再啰嗦几句，我们谈下这种编码的局限性。

这样编码有如下几个问题：

- 硬编码：局限就是只能识别预设的几种问法
- 使用了全局变量 df：全局变量引用增加了代码的复杂性

- 使用简单字符串匹配：这样会导致服务理解复杂的语义
- 缺失异常处理：局限就是当出现异常数据时，可能会导致程序崩溃

那如何改进呢？当下的解决方案就是实际编码时，我们通过大模型自动生成代码，比如让 AI 编码工具（国产 qoder 或者 claude code 帮我们生成 Pandas 程序代码，或者直接使用 NL2SQL 工具来替代这种硬编码逻辑。

在以后的课程介绍中，我们也会介绍 NL2SQL 或 Text2SQL 工具。

接下来我们再来看下第二个工具函数——绘图函数。它的作用是生成图表并保存。

上面介绍过的重复内容这里我就不赘述了，后续介绍也会稍微加快一点进度。

代码块

```
1  @tool
2  def plot_sales_data(chart_type: str = "line", metric: str = "sales") -> str:
3      """
4      生成销售数据可视化图表。
5
6      参数：
7          chart_type: 图表类型，可选'line'（折线图）或'bar'（柱状图）
8          metric: 要绘制的指标，可选'sales'（销售额）或'profit'（利润）
9
10     返回：
11         图表保存路径
12     """
13     global df
14     plt.figure(figsize=(10, 6))
15
16     if chart_type == "line":
17         # 按月聚合销售额/利润
18         monthly = df.groupby('month')[metric].sum()
19         monthly.plot(kind='line', marker='o')
20         plt.title(f"Monthly {metric.capitalize()} Trend")
21         plt.xlabel("Month")
22         plt.ylabel(metric.capitalize())
23     elif chart_type == "bar":
24         # 按地区聚合
25         regional = df.groupby('region')[metric].sum()
26         regional.plot(kind='bar')
27         plt.title(f"{metric.capitalize()} by Region")
28         plt.xlabel("Region")
29         plt.ylabel(metric.capitalize())
30     else:
31         return f"不支持的图表类型: {chart_type}"
32
```

```
33     img_path = f"sales_{metric}_{chart_type}.png"
34     plt.savefig(img_path)
35     plt.close()
36     return f"图表已保存为{img_path}"
```

函数 `plot_sales_data` 是图表生成工具函数：

代码块

```
1 def plot_sales_data(chart_type: str = "line", metric: str = "sales") -> str:
```

这里参数：

| 参数                      | 说明                                                                  |
|-------------------------|---------------------------------------------------------------------|
| <code>chart_type</code> | 表示图表类型，默认 <code>"line"</code> （折线图），你也可以选择 <code>"bar"</code> （柱状图） |
| <code>metric</code>     | 这是指标字段，默认 <code>"sales"</code> （销售额），可选 <code>"profit"</code> （利润）  |
| <code>-&gt; str</code>  | 返回保存的图表文件路径                                                         |

紧接着就是文档字符串。告诉大模型这个工具的用途、参数含义和可选值。大模型据此决定何时调用、传什么参数。

代码块

```
1     """
2     生成销售数据可视化图表。
3     ...
4     """
```

紧接着是函数 `figure`，作用是创建新图表：

代码块

```
1     plt.figure(figsize=(10, 6))
```

参数 `figsize`，作用是描绘画布尺寸，这里指定为宽10英寸，高6英寸。

| 参数 | 含义 |
|----|----|
|    |    |

```
figsize=(10, 6)
```

画布尺寸：宽10英寸，高6英寸

代码块

```
1         if chart_type == "line":
```

接下来就是条件逻辑，判断图表类型，如果图类型是折线图，那么按月进行分组聚合并汇总，指标列选择的是“销售量-sales”。总结便是按月汇总销售额。

代码块

```
1         monthly = df.groupby('month')[metric].sum()
```

下面 plot 函数就是绘制折线图函数。作用是按月绘制折线图，并且数据点采用圆圈标记。

代码块

```
1         monthly.plot(kind='line', marker='o')
```

| 参数                       | 含义       |
|--------------------------|----------|
| <code>kind='line'</code> | 折线图      |
| <code>marker='o'</code>  | 数据点用圆圈标记 |

接下来就是设置图表标题和轴标签：

代码块

```
1         plt.title(f"Monthly {metric.capitalize()} Trend")
2         plt.xlabel("Month")
3         plt.ylabel(metric.capitalize())
```

| 方法                        | 作用        |
|---------------------------|-----------|
| <code>plt.title()</code>  | 图表标题      |
| <code>plt.xlabel()</code> | X轴标签（月份）  |
| <code>plt.ylabel()</code> | Y轴标签（销售额） |
|                           |           |



|                            |                       |
|----------------------------|-----------------------|
| <code>.capitalize()</code> | 首字母大写 (sales → Sales) |
|----------------------------|-----------------------|

折线图条件逻辑结束后就是柱状图条件逻辑：

代码块

```
1     elif chart_type == "bar":
```

柱状图的逻辑不是根据月份绘制销售额，而是根据地区分组绘制柱状图，如下所示：

代码块

```
1         regional = df.groupby('region')[metric].sum()
2         regional.plot(kind='bar')
```

并把绘制好的柱状图生成图片，并保存到本地磁盘。

代码块

```
1     img_path = f"sales_{metric}_{chart_type}.png"
2     plt.savefig(img_path)
```

保存成功后，最后调用 close 函数关闭图表，释放内存。

代码块

```
1     plt.close()
```

整个图表生成函数的逻辑就结束了。这个函数工具的场景可以是：

| 用户请求       | 大模型调用                                         | 生成文件                              |
|------------|-----------------------------------------------|-----------------------------------|
| "画个销售额趋势图" | <code>plot_sales_data("line", "sales")</code> | <code>sales_sales_line.png</code> |
| "各地区利润柱状图" | <code>plot_sales_data("bar", "profit")</code> | <code>sales_profit_bar.png</code> |

保存数据函数工具结束之后，下一步我们介绍最后一个工具函数——分析工具函数。

工具3：分析工具。这也是相对其他两个函数而言最难的一个工具函数，它的作用是生成业务洞察。完整函数定义如下：

代码块

```
1  @tool
2  def analyze_sales_trend(question: str) -> str:
3      """
4      对销售数据进行深度分析，生成业务洞察。
5
6      参数：
7          question: 分析问题，如'为什么利润下降了？'
8
9      返回：
10         分析结果
11     """
12     global df
13     q = question.lower()
14
15     if "利润下降" in q or "利润为什么" in q:
16         # 按月份计算利润变化
17         monthly_profit = df.groupby('month')['profit'].sum()
18         if len(monthly_profit) >= 2:
19             months = list(monthly_profit.index)
20             changes = []
21             for i in range(1, len(months)):
22                 change = monthly_profit.iloc[i] - monthly_profit.iloc[i-1]
23                 changes.append(f"{months[i]}月相比{months[i-1]}月: {change}")
24             return f"利润变化趋势: \n" + "\n".join(changes)
25
26     return f"请提供更具体的分析问题。"
```

老习惯，我们还是简单解释下这个销售趋势分析工具函数的核心代码部分：

代码块

```
1  def analyze_sales_trend(question: str) -> str:
```

函数定义非常简单，就是接收用户提出的问题，经过处理后，返回字符串的分析结果。

接下来就是文档字符串说明。

代码块

```
1      """
2      对销售数据进行深度分析，生成业务洞察。
```

```
3      ...
4      .....
```

再往下就是意图识别板块，作用是检测用户是否在询问利润变化原因。

代码块

```
1      if "利润下降" in q or "利润为什么" in q:
```

接着就是按月分组聚合，进行利润求和：

代码块

```
1      monthly_profit = df.groupby('month')['profit'].sum()
```

再往下就是数据量检查，判断条件为至少要有2个月的数据才能比较变化趋势。

代码块

```
1      if len(monthly_profit) >= 2:
```

接着获取月份列表：如 ['Jan', 'Feb', 'Mar']。

代码块

```
1      months = list(monthly_profit.index)
```

然后遍历计算环比变化：从第2个月开始，与前一个月比较。

代码块

```
1      changes = []
2      for i in range(1, len(months)):
3          change = monthly_profit.iloc[i] - monthly_profit.iloc[i-1]
```

然后计算利润变化量，这里的 change 值为：利润变化，如果值为正，表示增长；如果值为负，则表示下降。

工作流程演示如下所示：

代码块

```
1  用户问: "为什么利润下降了? "  
2      ↓  
3  大模型识别为分析类问题 → 调用 analyze_sales_trend  
4      ↓  
5  检测关键词: "利润下降"  
6      ↓  
7  按月分组计算利润总和  
8      ↓  
9  环比计算: Feb-Jan=+20, Mar-Feb=+320  
10     ↓  
11  返回: "利润变化趋势: \nFeb月相比Jan月: 20\nMar月相比Feb月: 320"  
12     ↓  
13  大模型结合数据生成自然语言回答: "实际上利润没有下降, 而是持续增长..."
```

当然如果你想优化此数据洞察工具也可以, 比如: 你可以结合大模型进行真正的智能分析, 比如你可以提供你的数据和变化作为提示词, 然后发送给模型, 让模型给你更加人性化的分析和业务建议。

代码块

```
1  # 在函数内部调用 LLM 生成洞察  
2  analysis_prompt = f"""  
3  数据: {monthly_profit.to_dict()}  
4  变化: {changes}  
5  
6  请分析利润变化趋势, 并给出可能的原因和业务建议。  
7  """  
8  insight = model.invoke(analysis_prompt).content  
9  return insight
```

## 3.2 构建带记忆的Agent

上面我们花了比较大的篇幅介绍完如何定义工具, 接下来我们接着构建 Agent。这才是整个案例最核心的部分。这里我们采用 LangGraph 的 `StateGraph` 状态图来管理工作流节点的状态和记忆。

代码块

```
1  # 初始化模型  
2  model = ChatOpenAI(model="gpt-4", temperature=0)  
3  
4  # 创建带工具的Agent  
5  agent = create_agent(  
6      model=model,
```

```

7     tools=[query_sales_data, plot_sales_data, analyze_sales_trend],
8     system_prompt="""你是一个专业的数据分析助手。你可以：
9     1. 使用query_sales_data回答数据查询问题
10    2. 使用plot_sales_data生成图表
11    3. 使用analyze_sales_trend进行深度分析
12
13    回答要简洁、专业。"""
14 )
15
16 # 定义状态
17 class AnalyzerState(TypedDict):
18     messages: Annotated[list, add_messages]
19     context: dict
20
21 # 定义节点函数
22 def run_agent(state: AnalyzerState):
23     result = agent.invoke({"messages": state["messages"]})
24     return {"messages": [result["messages"][-1]]}
25
26 # 构建状态图
27 graph = StateGraph(AnalyzerState)
28 graph.add_node("agent", run_agent)
29 graph.set_entry_point("agent")
30 graph.add_edge("agent", END)
31
32 # 添加检查点实现记忆
33 checkpointer = InMemorySaver()
34 graph = graph.compile(checkpointer=checkpointer)

```

这段代码做了几件关键的事情：

1. **首先初始化模型：** 我们使用 `ChatOpenAI`，`temperature=0` 确保输出稳定，适合数据分析任务。
2. **接着创建 Agent：** `create_agent` 将模型和三个工具绑定在一起，并设置系统提示。系统提示告诉模型什么时候该用哪个工具。
3. **然后定义状态：** `AnalyzerState` 中的 `messages` 字段使用了 `add_messages` ——这是一个 reducer 函数，能自动把新消息追加到历史中。`context` 字段可以存储额外的上下文信息（如用户偏好、中间结果）。
4. **接下来定义节点函数：** `run_agent` 接收当前状态，调用 `agent.invoke` 处理所有消息，然后把最终回复作为新消息返回。注意，我们只取最后一条消息（模型的回答），而不是整个历史，避免重复存储。
5. **然后就是构建状态图，也就是创建工作流：** 这里只有一个节点 `agent`，从入口进入，执行完直接结束。这是最简单的 ReAct 模式。

6. 最后是添加检查点（记忆）：`InMemorySaver` 会在每次节点执行后保存状态。我们编译图时传入 `checkpointer`，这样多次调用 `invoke` 时，只要使用相同的 `thread_id`，状态就会自动恢复——从而实现多轮对话记忆。

**需要注意：**这里的关键是 `add_messages`：它保证每次更新 `messages` 时，不是覆盖，而是追加。所以对话历史会自然累积，不会丢失。

虽然这个图只有一个节点，但 `create_agent` 内部已经包含了完整的 ReAct 循环。模型会在需要时自动调用工具，我们不需要额外写条件边。

## 3.3 交互测试

### 3.3.1 【开场：见证“智能”的时刻】

好，工具定义好了，Agent 也构建完成了，记忆机制也配置好了。现在，我们进入**交互测试阶段**。

我们看看这个智能数据分析助手到底能不能理解老板的问题，能不能自动调用正确的工具，能不能记住我们之前聊了什么。

大家跟着我的思路，一起看代码，一起看效果。

### 3.3.2 【代码演示：交互循环的实现】

我们先实现一个简单的命令行交互循环。代码如下：

代码块

```
1  def chat():
2      print("=" * 50)
3      print("智能数据分析助手（LangChain版）")
4      print("输入 'exit' 退出")
5      print("=" * 50)
6
7      thread_id = "data_analyzer_001"
8      config = {"configurable": {"thread_id": thread_id}}
9
10     while True:
11         user_input = input("\n👤 你： ")
```

```

12         if user_input.lower() == 'exit':
13             break
14
15         result = graph.invoke(
16             {"messages": [{"role": "user", "content": user_input}]},
17             config=config
18         )
19         print(f"🤖 助手: {result['messages'][-1].content}")
20
21     if __name__ == "__main__":
22         chat()

```

下面我们简单解释下：

- `thread_id = "data_analyzer_001"`：给这个会话一个唯一ID。同一个ID下的所有对话会被自动保存和加载，实现多轮记忆。
- `config = {"configurable": {"thread_id": thread_id}}`：这是 LangGraph 要求的配置格式，告诉检查点存储器用哪个ID来保存/加载状态。
- `graph.invoke(...)`：每次用户输入，我们就调用状态图。传入当前用户消息和配置。图会自动加载历史状态，执行节点，保存新状态，最后返回结果。
- `result['messages'][-1]['content']`：从返回的消息列表中取出最后一条（即Agent的回答），打印出来。

**这里注意：**我们并不需要手动管理 `messages` 列表——因为 `graph.invoke` 会根据 `thread_id` 自动加载历史消息，并把新消息追加进去。这也是使用 LangChain 框架给我们带来的便利之处。

### 3.3.3 【演示效果：三类问题全覆盖】

好了，万事俱备，只欠东风。现在，我们来实际运行一下。模拟一次完整的对话，展示 Agent 如何回答三类不同的问题。

**模拟运行输出：**

代码块

```

1  =====
2  智能数据分析助手（LangChain版）
3  输入 'exit' 退出
4  =====
5
6  🧑 你：总销售额是多少？

```

程序执行后截图如下：

```
(venv) ai@aideMacBook 5-overall % python main.py
=====
智能数据分析助手（LangChain版）
输入 'exit' 退出
=====

👤 你：总销售额是多少？
🤖 助手：总销售额为12900。
```

第一个问题，用户问‘总销售额是多少’。当 Agent 看到这个问题，判断它属于**数据查询**类。模型内部思考先思考：‘用户想求和，应该调用 `query_sales_data` 工具。于是它调用工具，工具里的逻辑因为找到 `"总销售额"` 关键词，所以会返回 `df['sales'].sum()` 的结果。然后 Agent 再把数字包装成自然语言输出，并返回给我们。

#### 代码块

- 1 👤 你：销售额最高的月份是哪个月？
- 2 🤖 助手：销售额最高的记录是3月。

紧接着，我们问第二个问题，‘销售额最高的月份’。Agent 再次调用 `query_sales_data` 工具，工具里匹配到 `"最高"` 和 `"销售额"` 字样，于是定位到最大值的行，然后返回一个完整的句子。

执行结果如下图所示：

```
(venv) ai@aideMacBook 5-overall % python main.py
=====
智能数据分析助手（LangChain版）
输入 'exit' 退出
=====

👤 你：总销售额是多少？
🤖 助手：总销售额为12900。

👤 你：销售额最高的月份
🤖 助手：销售额最高的月份是3月。
```

这里需注意，这里并没有画图，因为 Agent 知道用户只要一个答案，并不是图表。



#### 代码块

- 1  你：画一张销售额的月度趋势图
- 2  助手：销售额的月度趋势图已经生成，文件名为sales\_sales\_line.png。

接着来到第三个问题，用户要求‘画一张销售额的月度趋势图’。这属于**数据可视化**类。Agent 识别出关键词‘画’和‘趋势图’，判断应该调用 `plot_sales_data` 工具。工具按月份聚合销售额，生成折线图，并保存为PNG文件，然后返回文件路径。注意，这里 Agent 没有输出数字，而是返回了文件路径——因为画图是它的任务，看图是用户的事。

代码成功执行后截图如下：

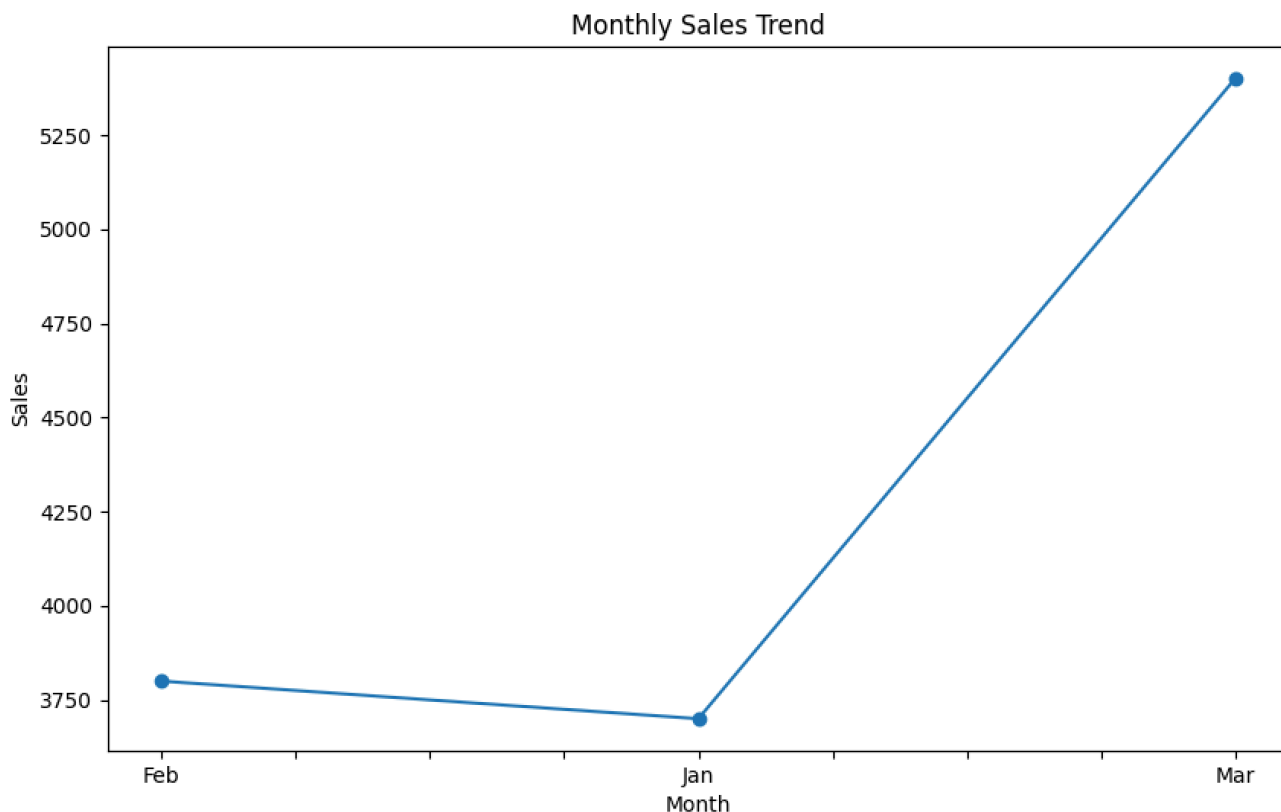
```
[(venv) ai@aideMacBook 5-overall % python main.py
=====
智能数据分析助手（LangChain版）
输入 'exit' 退出
=====

👤 你：总销售额是多少？
🤖 助手：总销售额为12900。

👤 你：销售额最高的月份是哪个月？
🤖 助手：销售额最高的月份是3月。

👤 你：画一张销售额的月度趋势图
🤖 助手：销售额的月度趋势图已经生成，文件名为 sales_sales_line.png。
```

查看图片效果为：



#### 代码块

- 1  你：为什么利润下降了？
- 2  助手：利润下降可能是由于销售额的减少或者成本的增加。
- 3 从我们的数据中可以看出，1月份相比2月份，利润下降了20，而3月份相比1月份，利润又上升了340。
- 4 这可能是由于在这些月份中，销售额和成本发生了变化。具体的原因需要进一步分析销售额和成本数据。

最后，我们进入第四个问题，‘为什么利润下降了？’这属于**多步分析**类。Agent调用 `analyze_sales_trend` 工具，工具按月份计算利润变化，返回两个变化值。注意，这里Agent没有做‘原因’分析（比如因为销量下降或成本上升），它只是计算了变化量。真正的因果分析需要结合更多数据和外部知识，但至少它提供了数据基础。

程序执行成功后截图如下：

 你：为什么利润下降了  
 助手：利润下降可能是由于销售额的减少或者成本的增加。从我们的数据中可以看出，1月份相比2月份，利润下降了20，而3月份相比1月份，利润又上升了340。这可能是由于在这些月份中，销售额和成本发生了变化。具体的原因需要进一步分析销售额和成本数据。

### 3.3.4 【记忆功能的验证】

不知道大家有没有注意到，在整个对话中，我们**并没有重复任何上下文**。Agent能连续回答不同的问题，全靠记忆。

我们再来验证一下记忆是否真的生效。假设我们继续问，模型返回如下结果：

代码块

- 1  你：那利润最高的地区是哪里？
- 2  助手：利润最高的地区是East

由此可知，Agent 仍然还记得我们之前在讨论‘利润’这个话题。如果没有记忆，它可能会反问：‘你说的那是指什么？’

这就是 `thread_id` + `checkpoint` 的作用。每次 `graph.invoke` 都会加载之前的状态，所以 Agent 知道历史对话。你可以在对话中途关闭程序，再重新运行，只要使用同一个 `thread_id`，状态仍然存在。我们这里因为用的是 `InMemorySaver`，属于内存级存储，当进程重启后记忆会丢失，所以生产环境大家可以换成 `SqliteSaver` 或 `PostgresSaver`。

### 3.3.5 【模型自主决策的核心】

LangChain 框架编码的程序运行已经到此为止。

我们发现，在整个执行过程中，最精彩的部分是：我们从来没有告诉 Agent ‘什么时候应该用哪个工具’。

- 比如当用户问‘总销售额’，模型会自动调用 `query_sales_data`。
- 比如用户说‘画一张图’，模型会自动调用 `plot_sales_data`。
- 比如用户问‘为什么利润下降了’，模型也会自动调用 `analyze_sales_trend`。

这是怎么做到的？我们前面已经提醒过了，靠的就是工具的文档字符串和系统提示，我们再次强调。

每个 `@tool` 函数的文档字符串（即 `"""..."""` 部分）它会被 LangChain 自动转换成模型能看到的函数描述。模型根据这些描述和用户问题的语义，自主决定调用哪个工具、填什么参数。

这就是 **Function Calling + ReAct** 的威力：模型不是按照传统 if-else 执行，而是执行动态推理。

### 3.3.6 【小结与常见问题】

好了，LangChain 交互测试我们就演示到这里。我们再来总结一下这个 Agent 的三个核心能力：

1. **第一：工具调用。**智能体 **Agent** 能根据问题类型自动调用数据查询、绘图、分析工具。
2. **第二：历史记忆。**智能体 **Agent** 能记住对话历史，支持‘那利润呢’这样的指代。
3. **第三：多轮会话。**能连续回答不同问题，不需要用户重复上下文。

当然实际操作中，我们也会碰到一些常见问题，这里也做一个提醒：

- **第一：底层模型能力不同，会导致 Agent 不调用正确的工具。**如果碰到这样的问题，检查工具的文档字符串是否清晰，描述要写清楚‘什么时候调用’，比如‘当用户询问总销售额时’。
- **第二：如果记忆不生效：**检查是否在 `compile` 时传了 `checkpointers`，并且每次 `invoke` 时传了相同的 `thread_id`。
- **第三：如果Agent回答太啰嗦：**调整 `system_prompt`，加上‘回答要简洁’。

这个例子大家线下要亲自实操，仔细消化下 LangChain 智能体 Agent 底层的编码和运转机制。

下面为了强化和提升大家的认知，我们稍微提高下技术难度。

我们将进入一个**进阶实现**——用自定义 StateGraph 实现 Plan-and-Execute 模式，让 Agent 能够先规划后再执行，处理更加复杂的任务。

下面我们开始。

### 3.4 进阶：自定义 StateGraph 实现 Plan-and-Execute

上面我们用了 `create_agent` + 单节点状态图，已经实现了一个功能完善的数据分析助手。

但如果我们想要更精细的控制——比如先规划后执行、动态重规划、或者人机协同——那就需要自定义 StateGraph。

这就是 Plan-and-Execute 模式。这个模式可以将规划和执行分离，它特别适合多步骤、可预见的任务。

#### 3.4.1 【设计思想回顾】

编码前，我们还是先快速回顾一下 Plan-and-Execute 的核心思想，它有两个阶段：

- **第一：规划阶段。**模型接收用户目标，生成一个步骤列表。
- **第二：执行阶段。**按照步骤列表顺序执行，可以串行或并行。

我们今天因为时间原因实现一个简化版：我们用 `planner` 节点生成计划，用 `executor` 节点依次执行每一步。执行时仍然复用我们之前定义的 `agent`（那个带工具的 ReAct Agent）。

我们在什么场景下需要这种模式？因为有些场景下，用户喜欢一次性提出了多个子任务，比如：‘帮我分析销售数据，包括总销售额、最高利润月份，并画一张利润趋势图’。如果采用 ReAct 模式，模

型可能会串行地一个个执行，但步骤之间没有显式的规划。如果我们使用 Plan-and-Execute，可以先生成计划列表，然后按计划执行，这样会更加可控。

### 3.4.2 【代码实现】

好，下面我们完成 Plan-and-Execute 代码。

#### 第一步：我们先定义状态

代码块

```
1  from langgraph.graph import StateGraph, END
2  from typing import List, Tuple
3
4  class PlanState(TypedDict):
5      input: str                # 用户原始输入
6      plan: List[str]           # 步骤列表，如['计算总销售额', '查找最高利润
   月份', '画利润趋势图']
7      past_steps: List[Tuple[str, str]] # 已执行步骤及结果
8      final_answer: str         # 最终答案
```

这里定义了一个 `PlanState` 类型，用于在 LangGraph 中的状态管理。这是一个非常典型的 ReAct / Plan-and-Execute 模式的状态结构。这个 `PlanState` 类型的结构包括了4个字段，其中：

| 字段                        | 类型                                 | 用途                 |
|---------------------------|------------------------------------|--------------------|
| <code>input</code>        | <code>str</code>                   | 存储用户的原始输入/问题       |
| <code>plan</code>         | <code>List[str]</code>             | 待执行的步骤列表，这在规划阶段生成  |
| <code>past_steps</code>   | <code>List[Tuple[str, str]]</code> | 已完成的步骤及对应结果，用于追踪进度 |
| <code>final_answer</code> | <code>str</code>                   | 最终返回给用户的答案         |

#### 第二步：规划节点

规划节点就是负责把用户需求拆解成具体要执行的步骤。我们用一个简单的Prompt让模型生成列表。

代码块

```
1 def planner(state: PlanState):
2     """规划节点：生成执行计划"""
3     prompt = f"为以下任务生成步骤列表: {state['input']}"
4     response = model.invoke(prompt)
5     lines = response.content.strip().split('\n')
6     # 过滤空行和标题行，保留步骤
7     plan = [line.strip() for line in lines if line.strip() and not
8             line.startswith('步骤')]
9     return {"plan": plan}
```

下面简单解释这个 `planner` 函数：

代码块

```
1 def planner(state: PlanState):
```

Planner 函数会接收一个 `PlanState` 类型的状态字典作为参数，它的作用是“根据用户提问，生成执行计划”。

代码块

```
1     prompt = f"为以下任务生成步骤列表: {state['input']}"
```

接下来是**构建 Prompt**。我们从状态中提取用户的原始输入 `state['input']`，拼接成提示词发送给 LLM，LLM 处理完毕后，返回结果。

代码块

```
1     response = model.invoke(prompt)
```

这里说明下，`response` 变量是模型返回的 `AIMessage` 类型对象，而 `response.content` 则是返回模型生成的文本内容。这在我们处理具体的编码时容易混淆。

| 变量                            | 示例值                             |
|-------------------------------|---------------------------------|
| <code>response</code>         | 模型返回的 <code>AIMessage</code> 对象 |
| <code>response.content</code> | 模型生成的文本内容                       |

接着这条复杂的语句就是将模型返回的多行文本按行进行分割，拆分成任务列表。

```
代码块 lines = response.content.strip().split('\n')
```

**按行分割：**将模型返回的多行文本拆分成列表，下面就是一个将原始内容分割后的样例。

代码块

```
1  原始内容：
2  1. 读取销售数据文件
3  2. 计算每月利润
4  3. 找出最高利润月份
5  4. 绘制利润趋势图
6
7  分割后：
8  ['1. 读取销售数据文件', '2. 计算每月利润', '3. 找出最高利润月份', '4. 绘制利润趋势图']
```

**接下来的 for 循环使用列表推导式过滤，遍历每行，只保留有效步骤。**

代码块

```
1  # 过滤空行和标题行，保留步骤
2  plan = [line.strip() for line in lines if line.strip() and not
           line.startswith('步骤')]
```

其中：

| 条件                                     | 作用             | 示例                        |
|----------------------------------------|----------------|---------------------------|
| <code>line.strip()</code>              | 去除首尾空格，且判断是否非空 | <code>''</code> → 过滤掉     |
| <code>not line.startswith('步骤')</code> | 过滤以"步骤"开头的标题行  | <code>'步骤一：'</code> → 过滤掉 |

**这里也给出一个过滤的示例：**

代码块

```
1  输入：['', '步骤：', '1. 读取数据', ' ', '2. 计算利润']
2  输出：['1. 读取数据', '2. 计算利润']
```

**最后返回规划结果：**

代码块

```
1 return {"plan": plan}
```

比如：

| 返回前 state                                 | 返回后 state                                                     |
|-------------------------------------------|---------------------------------------------------------------|
| <code>{"input": "...", "plan": []}</code> | <code>{"input": "...", "plan": ["1. 读取数据", "2. 计算利润"]}</code> |

下面就是该过程的完整数据流描述：

代码块

```
1  用户输入
2      ↓
3  state = {"input": "分析销售数据", "plan": [], "past_steps": [], "final_answer": ""}
4      ↓
5  planner(state)
6      |—— 构建 prompt: "为以下任务生成步骤列表：分析销售数据"
7      |—— 调用 LLM → 返回多行文本
8      |—— 分割 → 过滤 → 得到 ["读取数据", "计算利润", "画图"]
9      |—— 返回 {"plan": ["读取数据", "计算利润", "画图"]}
10     ↓
11  state 更新为：
12  {"input": "分析销售数据", "plan": ["读取数据", "计算利润", "画图"], ...}
```

### 第三步：执行节点

介绍完规划节点后，下一步我们准备执行节点逻辑。

代码块

```
1 def executor(state: PlanState):
2     """执行节点：按顺序执行步骤"""
3     if not state["plan"]:
4         return {}
5
6     step = state["plan"][0]
7     # 调用Agent执行当前步骤
8     result = agent.invoke({
9         "messages": [{"role": "user", "content": f"执行：{step}"}]
10    })
```



```

11     result_content = result["messages"][-1].content
12
13     # 安全更新 past_steps (创建新列表, 避免副作用)
14     new_past_steps = state.get("past_steps", []) + [(step, result_content)]
15     remaining_plan = state["plan"][1:]
16
17     # 关键修复: 最后一步时综合所有步骤结果
18     if not remaining_plan:
19         # 生成完整总结
20         summary_lines = []
21         for s, r in new_past_steps:
22             summary_lines.append(f"{s}: {r}")
23         final_answer = "\n".join(summary_lines)
24         print(f"🎯 生成最终答案")
25     else:
26         final_answer = ""
27
28     return {
29         "plan": remaining_plan,
30         "past_steps": new_past_steps,
31         "final_answer": final_answer
32     }

```

下面我们简单介绍下该执行节点逻辑，这个执行节点方法为 `executor`，它会接收 `PlanState` 状态字典，负责执行计划中的下一步。

代码块

```

1     def executor(state: PlanState):

```

该执行器方法的文档字符串为：这是执行节点，按顺序执行计划步骤。

代码块

```

1         """执行节点：按顺序执行步骤"""

```

接下来就是检查计划是否为空：如果没有待执行步骤，返回空字典（不更新状态）。

代码块

```

1         if not state["plan"]:
2             return {}

```

然后**获取当前步骤**：取出计划列表的第一个步骤。

代码块

```
1      step = state["plan"][0]
```

然后**调用 Agent**：将当前步骤包装成消息，发送给 ReAct Agent 执行。

代码块

```
1      # 调用Agent执行当前步骤
2      result = agent.invoke({
3          "messages": [{ "role": "user", "content": f"执行: {step}" }]
4      })
```

接着**提取结果**：从 Agent 返回的消息列表中，取最后一条AI 回复的内容。

代码块

```
1      result_content = result["messages"][-1].content
```

然后创建新的历史记录列表：

代码块

```
1      new_past_steps = state.get("past_steps", []) + [(step, result_content)]
```

| 部分                          | 含义                       |
|-----------------------------|--------------------------|
| state.get("past_steps", []) | 获取当前已执行步骤，如果不存在则返回空列表 [] |
| + [(step, result_content)]  | 将当前步骤和结果打包成元组，追加到列表末尾    |
| new_past_steps              | 全新的列表，包含所有历史 + 当前步骤      |

比如：

代码块

```
1  执行前: past_steps = [("步骤1", "结果1")]
2  当前: step = "步骤2", result_content = "结果2"
```

3 执行后：new\_past\_steps = [("步骤1", "结果1"), ("步骤2", "结果2")]

接下来就是获取剩余计划操作：

代码块

```
1 remaining_plan = state["plan"][1:]
```

这里：

| 代码             | 含义                             |
|----------------|--------------------------------|
| state["plan"]  | 当前计划列表，如 ["查销售额", "查利润", "画图"] |
| [1:]           | 切片，去掉第1个元素（已执行的步骤）             |
| remaining_plan | 剩余计划，如 ["查利润", "画图"]           |

比如：

代码块

```
1 执行前：plan = ["查销售额", "查利润", "画图"]
2 执行后：remaining_plan = ["查利润", "画图"] ← 去掉了"查销售额"
```

然后判断是否还有剩余步骤：

| 条件                 | 含义                        |
|--------------------|---------------------------|
| not remaining_plan | remaining_plan 为空列表（最后一步） |
| remaining_plan 有值  | 还有后续步骤                    |

最后一步：汇总所有结果：

| 步骤 | 操作                  |
|----|---------------------|
| 1  | 创建空列表 summary_lines |

|   |                                                      |
|---|------------------------------------------------------|
| 2 | 遍历 <code>new_past_steps</code> （所有步骤和结果）             |
| 3 | 每步格式化为 <code>"步骤： 结果"</code>                         |
| 4 | 用 <code>\n</code> 连接所有行，生成 <code>final_answer</code> |

至此，执行节点逻辑声明完毕。

### 第四步：构建图

执行器节点完成之后，下一步我们就要构建整个工作流图。

代码块

```
1  # 添加条件判断函数
2  def should_continue(state: PlanState) -> str:
3      """判断是否需要继续执行"""
4      if state.get("plan"): # 还有剩余步骤
5          return "continue"
6      return "end"
7
8  # 构建图
9  plan_graph = StateGraph(PlanState)
10 # # 添加节点
11 plan_graph.add_node("planner", planner)
12 plan_graph.add_node("executor", executor)
13
14 # 设置入口
15 plan_graph.set_entry_point("planner")
16
17 # 设置边：从计划器 → 执行器
18 plan_graph.add_edge("planner", "executor")
19
20 # 关键修改：添加条件循环边
21 plan_graph.add_conditional_edges(
22     "executor",
23     should_continue,
24     {
25         "continue": "executor", # 还有步骤，回到 executor 继续
26         "end": END # 没步骤了，结束
27     }
28 )
29
30 plan_app = plan_graph.compile()
```

## 第五步：测试

流程图已经编写完毕，下一步我们可以运行测试下实际效果了，通过创建智能体，访问大模型，调用工具，并按照我们设计的流程一次输出全部问题结果。

代码块

```
1  result = plan_app.invoke({
2      "input": "帮我分析销售数据，包括总销售额、最高利润月份，并画一张利润趋势图",
3      "plan": [],
4      "past_steps": [],
5      "final_answer": ""
6  })
7
8  print(f"\n{'='*50}")
9  print(f"🎯 最终结果：")
10 print(result['final_answer'])
```

运行后，你会看到类似输出：

代码块

```
1  🎯 生成最终答案
2
3  =====
4  🎯 最终结果：
5  首先，你需要收集所有相关的销售数据。这可能包括每月的销售额、成本、利润等。确保你有足够的数
   据来进行全面的分析。：对不起，我可能误解了您的指示。作为一个数据分析助手，我无法收集数据，
   但我可以帮助您分析和解释已有的数据。如果您有任何关于销售数据的问题，例如查询最高销售额的月
   份，或者需要生成销售数据的图表，或者需要深度分析销售趋势，我都可以帮助您。请提供具体的分析
   需求。
6  将收集到的数据整理成一个易于分析的格式。这可能包括删除重复项、填充缺失值、转换数据类型等。
   你可能需要使用电子表格或数据分析软件来完成这个步骤。：对不起，我无法执行数据清理和预处理任
   务。我是一个数据分析助手，我可以帮助你查询销售数据、生成销售数据图表和进行销售趋势分析。你
   可以向我提问，例如"销售额最高的月份是什么？"或者"生成销售额的折线图"等。
7  将所有月份的销售额相加，得到总销售额。这将给你一个整体的销售情况。：总销售额为12900。
8
9  通过比较每个月的利润，找出利润最高的月份。这将帮助你了解哪个月份的销售表现最好。：利润最高
   的月份是3月，主要由东部地区的高利润贡献，利润达到400。
10 使用图表来表示每个月的利润。这将帮助你更直观地看到销售利润的变化趋势。你可以使用柱状图、折
   线图或其他类型的图表，根据你的需要选择。：已经生成了表示每个月利润的折线图，图表保存为
   "sales_profit_line.png"。你可以查看这个图表以了解销售利润的变化趋势。
11
```

- 12

根据你的计算和图表，分析销售数据的结果。这可能包括解释为什么某个月份的利润最高，或者销售额的变化趋势等。：对不起，我需要更具体的问题来进行分析。例如，你可以问我"为什么3月的利润最高？"或者"销售额的变化趋势是怎样的？"。
- 13

将你的分析结果和观察写成报告。确保报告清晰、准确，并包含所有重要的信息。：抱歉，我需要更具体的信息才能进行分析并编写报告。例如，你可能想要知道哪个月的销售最高，或者你可能想要一个关于各地区销售额的柱状图，或者你可能想要深入分析为什么某个季度的利润下降了。请提供更多的细节，我将很高兴为你提供帮助。
- 14

将你的报告分享给相关的人员，如销售团队、管理层等。讨论你的发现，以及可能的改进策略。：抱歉，我作为一个数据分析助手，我无法直接分享报告或与团队进行讨论。但我可以帮助你生成报告和分析，你可以根据这些信息进行分享和讨论。如果你需要进行某种分析或生成报告，请告诉我具体的需求。

至此，Plan-and-Execute 模式的智能体开发结束。在该环节下，我们在：

- 规划节点先生成了类似 ['计算总销售额', '查找最高利润月份', '画利润趋势图'] 的整个规划列表。
- 然后在执行节点依次调用 agent 执行每个步骤。
- 最终答案由大模型根据智能体的多次调用结果汇总后输出。

3.4.3 【与ReAct模式的对比】

下面我们看看这个自定义 StateGraph 和之前的单节点 ReAct 模式有什么区别？

| 维度     | ReAct（单节点）           | Plan-and-Execute（自定义） |
|--------|----------------------|-----------------------|
| 决策时机   | 每步动态决策               | 预先规划，按步执行             |
| 模型调用次数 | 每个子任务都需多次模型调用（思考+工具） | 规划一次 + 每个子任务调用一次agent |
| 流程透明度  | 黑盒，依赖模型内部推理          | 白盒，计划列表清晰可见           |
| 灵活性    | 高，可随时调整              | 中，计划出错需重规划            |
| 适用场景   | 步骤不确定、需要动态调整的任务      | 步骤可预见的复合任务            |

这里我们再回忆下，到底什么时候用 Plan-and-Execute？

- 用户一次性提出多个明确子任务
- 子任务之间没有强依赖（或依赖简单）
- 希望控制执行顺序，便于监控和调试

### 3.4.4 【扩展思考】

当然，上面这个实现还有很多可以改进的地方：

1. **提示词优化**：我们可以让模型输出JSON格式的计划，比如 `{"steps": ["step1", "step2"]}`，标准化的 JSON 格式解析会更加稳定。
2. **并行执行**：对于没有依赖关系的步骤，我们不是采用顺序执行，而是可以设计成多个并行执行节点，这样提高会更高效一些。

上面这些进阶的技巧我们可以在后续的 LangGraph 高级课程中详细讲解。

好了，Plan-and-Execute的实现也结束了。

下面，我们再看看**基于 OpenClaw Skills 针对“数据分析助手”功能的技术实现方案**，看看同样的问题如何用配置化的方式解决，最终再横向对比下两种方案各自的特点。

## 第四部分：方案B——基于 OpenClaw Skills 的实现

### 4.1 OpenClaw 技能设计

OpenClaw 的核心是‘技能’（Skill）。每个技能就是一个独立的工具模块。这里 OpenClaw 我采用的模型是 DeepSeek-Reasoner 模型。

Skills 包含两部分：

1. `SKILL.md`：用自然语言描述希望调用工具的功能、参数和触发条件。
2. `__init__.py`：Python 实现，是用来真正干活的代码。

这种设计方式非常清晰和简洁，它可以让工具像 App 一样被安装、卸载以及共享。

下面，我们开始使用 OpenClaw 完成我们数据分析智能助手。

#### 4.1.1 【步骤1：创建技能目录】

OpenClaw 安装我们在第一节课已经进行了介绍，大家如果忘记的话，可以翻看以前的课程文档进行回顾。

首先，进入 OpenClaw 的技能目录，创建一个名为 `data_analyzer` 的文件夹，并创建两个空文件。

代码块

```
1  mkdir -p ~/.openclaw/skills
2  cd ~/.openclaw/skills/
3  mkdir data_analyzer
4  cd data_analyzer
5  touch SKILL.md __init__.py
```

执行成功后截图如下：

```
[ai@aideMacBook ~ % tree ~/.openclaw/skills
/Users/ai/.openclaw/skills
└─ data_analyzer
   └─ __init__.py
      SKILL.md

2 directories, 2 files
```

这里需要注意：技能目录的路径取决于你实际的 OpenClaw 安装位置。默认路径地址是 `~/.openclaw/skills/`。

#### 4.1.2 【步骤2：编写SKILL.md——用自然语言定义技能】

下面我们打开 `SKILL.md` 文件，写入以下内容。这是技能的‘身份证’，OpenClaw 的模型就是靠它来理解这个技能能做什么。

代码块

```
1  ---
2  name: data_analyzer
3  description: 销售数据分析技能，支持查询、图表生成和深度分析
4  version: 1.0.0
5  ---
6
```



```

7   # data_analyzer 技能
8
9   这个技能提供以下数据分析工具：
10
11  ## query_sales_data
12  对销售数据执行分析查询。支持：最高销售额/利润、总销售额/总利润、各地区/各月份统计、产品销
    量筛选。
13
14  参数：question (string) 用户的分析问题，如"销售额最高的月份是哪个月？"
15
16  ## plot_sales_data
17  生成销售数据可视化图表。
18
19  参数：
20  - chart_type (string) 图表类型：'line'折线图 或 'bar'柱状图
21  - metric (string) 指标：'sales'销售额 或 'profit'利润
22
23  ## analyze_sales_trend
24  对销售数据进行深度分析，生成业务洞察。
25
26  参数：question (string) 分析问题，如"为什么利润下降了？"

```

我们发现 `SKILL.md` 中的描述完全是自然语言。OpenClaw 在实际运行时会把这段描述注入到系统提示中，模型通过阅读这段文字就能知道：什么时候该调用 `query_sales_data`，什么时候该调用 `plot_sales_data`，每个参数是什么意思。这种**配置优先**的设计，可以让非技术人员也能添加新技能。

#### 4.1.3 【步骤3：编写 `__init__.py`——实现技能逻辑】

接下来，我们实现真正的Python代码。打开 `__init__.py`，写入以下内容。

代码块

```

1  from openclaw.skill import Skill
2  import pandas as pd
3  import matplotlib.pyplot as plt
4  import os
5
6  # 加载数据（技能启动时加载一次，避免重复IO）
7  df = pd.read_csv(os.path.join(os.path.dirname(__file__), "sales.csv"))
8
9  class DataAnalyzerSkill(Skill):
10     def __init__(self):

```

```

11         super().__init__()
12         self.df = df
13
14     def query_sales_data(self, question: str) -> str:
15         """数据查询实现"""
16         q = question.lower()
17
18         if "最高" in q or "最大" in q:
19             if "销售额" in q:
20                 max_row = self.df.loc[self.df['sales'].idxmax()]
21                 return f"销售额最高的记录是{max_row['month']}月，
22 {max_row['region']}地区，{max_row['product']}产品，销售额为{max_row['sales']}"
23             elif "利润" in q:
24                 max_row = self.df.loc[self.df['profit'].idxmax()]
25                 return f"利润最高的记录是{max_row['month']}月，{max_row['region']}
26 地区，利润为{max_row['profit']}"
27
28         if "总销售额" in q:
29             total = self.df['sales'].sum()
30             return f"总销售额为{total}"
31
32         if "总利润" in q:
33             total = self.df['profit'].sum()
34             return f"总利润为{total}"
35
36         return f"无法理解: {question}"
37
38     def plot_sales_data(self, chart_type: str = "line", metric: str = "sales")
39     -> str:
40         """绘图实现"""
41         plt.figure(figsize=(10, 6))
42
43         if chart_type == "line":
44             monthly = self.df.groupby('month')[metric].sum()
45             monthly.plot(kind='line', marker='o')
46             plt.title(f"Monthly {metric.capitalize()} Trend")
47         elif chart_type == "bar":
48             regional = self.df.groupby('region')[metric].sum()
49             regional.plot(kind='bar')
50             plt.title(f"{metric.capitalize()} by Region")
51         else:
52             return f"不支持的图表类型: {chart_type}"
53
54         img_path = f"/tmp/sales_{metric}_{chart_type}.png"
55         plt.savefig(img_path)
56         plt.close()
57         return f"图表已保存为{img_path}"

```

```

55
56     def analyze_sales_trend(self, question: str) -> str:
57         """分析实现"""
58         q = question.lower()
59
60         if "利润下降" in q or "利润为什么" in q:
61             monthly_profit = self.df.groupby('month')['profit'].sum()
62             if len(monthly_profit) >= 2:
63                 months = list(monthly_profit.index)
64                 changes = []
65                 for i in range(1, len(months)):
66                     change = monthly_profit.iloc[i] - monthly_profit.iloc[i-1]
67                     changes.append(f"{months[i]}月相比{months[i-1]}月:
68 {change}")
69
70                 return f"利润变化趋势: \n" + "\n".join(changes)
71
72         return f"请提供更具体的分析问题。"
73
74 # 注册技能
75 skill = DataAnalyzerSkill()

```

这里简单介绍下代码：

- **类 DataAnalyzerSkill 继承自 Skill**，如果我们使用 OpenClaw，那么每个技能类都必须继承 `openclaw.skill.Skill`。
- **\_\_init\_\_ 中加载数据**：技能启动时加载一次 CSV，这样就可以避免每次调用都读文件。我们这里调用了 LangChain 数据智能助手生成的销售 CSV 文件。
- **工具方法我们已经耳熟能详了**，这里工具函数的声明必须与 `SKILL.md` 中的方法工具名保持一一对应，比如：`query_sales_data`、`plot_sales_data`、`analyze_sales_trend`。OpenClaw 会根据模型输出的技能调用指令，从而调用对应的方法。
- **最后实例化类 DataAnalyzerSkill，并把它赋值给 skill 变量**：这是 OpenClaw 的编码约定，框架会扫描每个技能目录，找到名为 `skill` 的实例并进行加载。

#### 4.1.4 【强调与总结】

好了，我们的技能就已经写好了，是不是很简单，完全不像 LangChain 编写状态工作流那么繁琐。因为，`SKILL.md` 中的描述和 `__init__.py` 中的实现是完全分离的。这带来了几个好处：

1. **即使是非技术人员，也可以写 SKILL.md**：产品经理可以用自然语言描述一个新技能，工程师只需要实现对应的方法即可，工种、角色自然分离。

- 2. **技能可共享**：把整个 `data_analyzer` 文件夹打包，别人只需要把该文件夹进行打包，并放到他的技能目录就直接能用。
- 3. **热加载**：修改 `__init__.py` 后重启 OpenClaw 即可，不影响其他技能。

**下一步**：我们需要配置 OpenClaw 加载这个 Skill 技能即可，我们先复制销售 CSV 数据文件到 OpenClaw 技能目录。然后重启服务，就可以在聊天中进行测试了。

代码块

```
1  # 复制CSV文件到技能目录
2  cp /path/to/sales.csv ~/.openclaw/skills/data_analyzer/
3
4  # 注册 skills
5  openclaw config set skills.load.extraDirs
6    ["~/.openclaw/skills/data_analyzer"]
7
8  # 重启 openclaw gateway, 使自定义 skills 生效
9  openclaw gateway restart
10
11 # 验证技能是否加载
12 openclaw skills list
```

如果技能加载成功，控制台会将其状态标识为 ready，如下所示：

ai@aideMacBook skills % openclaw skills list

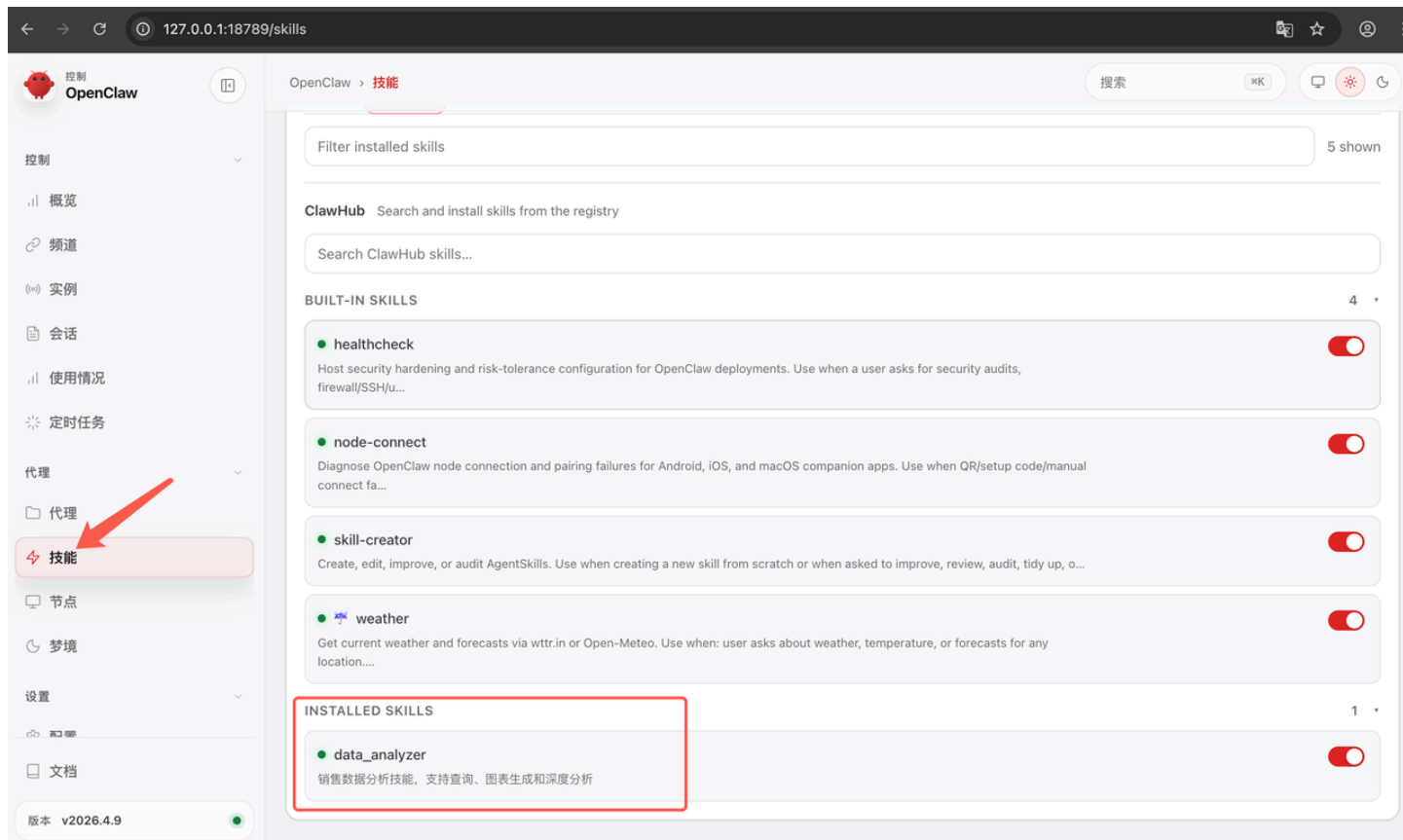
🦜 OpenClaw 2026.4.9 (0512059) – I'll butter your workflow like a lobster roll: messy, delicious, effective.

Skills (5/51 ready)

| Status        | Skill             | Description                                                                                                                                                                                                 | Source           |
|---------------|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|
| ✓ ready       | 📁 data_analyzer   | 销售数据分析技能，支持查询、图表生成和深度分析                                                                                                                                                                                     | openclaw-managed |
| ⚠ needs setup | 🔑 1password       | Set up and use 1Password CLI (op). Use when installing the CLI, enabling desktop app integration, signing in (single or multi-account), or reading/injecting/running secrets via op.                        | openclaw-bundled |
| ⚠ needs setup | 📝 apple-notes     | Manage Apple Notes via the `memo` CLI on macOS (create, view, edit, delete, search, move, and export notes). Use when a user asks OpenClaw to add a note, list notes, search notes, or manage note folders. | openclaw-bundled |
| ⚠ needs setup | 🕒 apple-reminders | Manage Apple Reminders via remindctl CLI (list, add, edit, complete, delete). Supports lists, date                                                                                                          | openclaw-bundled |

如果出现权限错误，可以尝试加 `sudo` ，或者检查 `~/.openclaw/` 目录的归属。

当然你也可以通过 openclaw dashboard 的 webUI 查看 skills 的安装情况，如截图所示：



好，紧接着我们需要重启 Openclaw，重启 openclaw 成功后，下一步，我们会实际测试这个技能的效果，并与 LangChain 方案进行对比。

#### 4.1.5 【验证与测试】

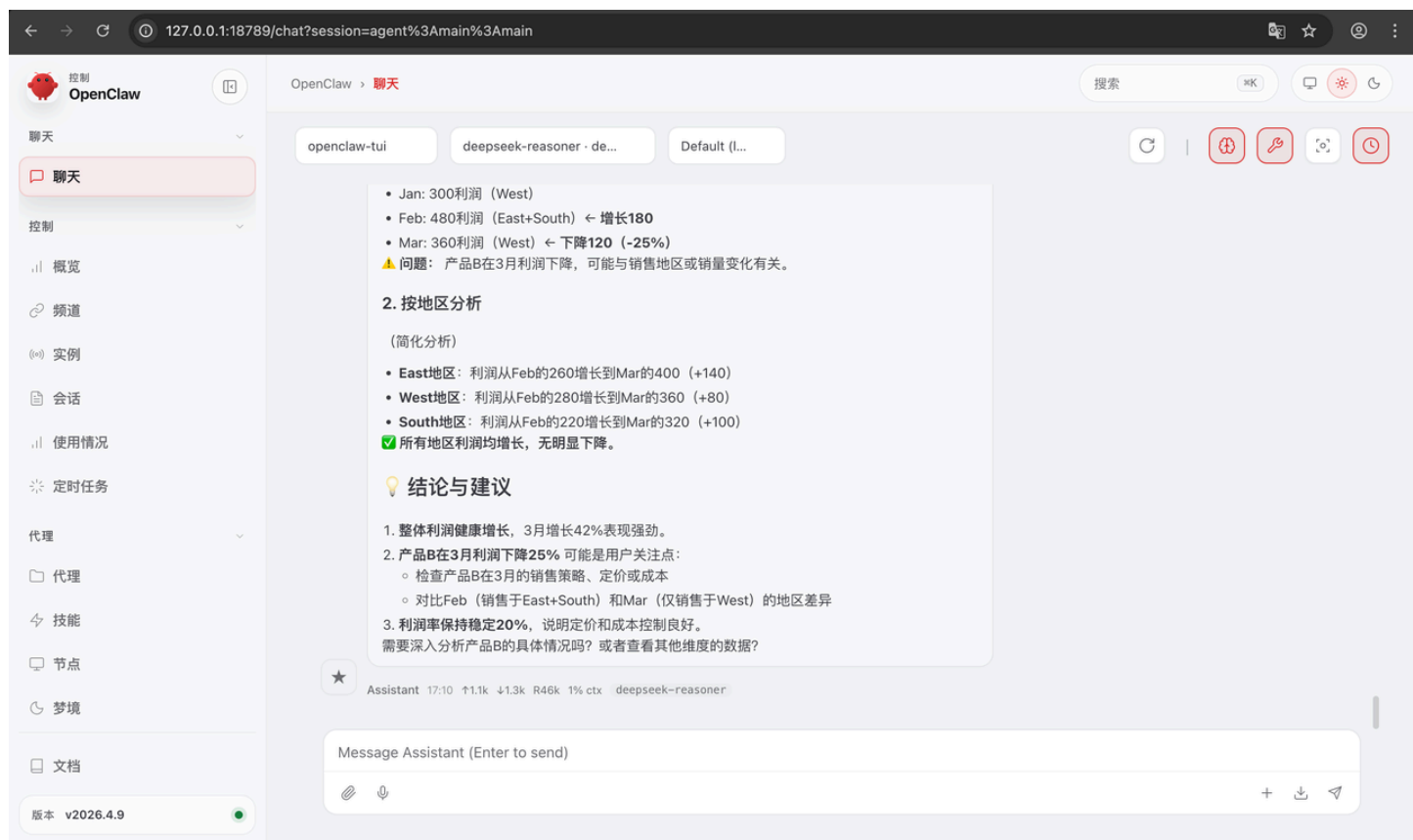
为了确认技能真的可以工作，我们这里可以简单测试一下。打开 OpenClaw 的聊天界面（可以是命令行、Web UI或接入的聊天平台）。

指令如下：

代码块

```
1 openclaw dashboard
```

指令如果执行成功，会打开 OpenClaw 的 WEB UI，如下图所示：



## 4.2 交互测试

好，技能配置好了，服务也重启了。现在，我们来实际测试一下这个OpenClaw技能到底能不能用。

打开OpenClaw的聊天界面——可以是命令行，也可以是Web UI，或者你接入的飞书、Discord。我这边用命令行演示。

### 4.2.1 【第一轮：数据查询】

先问一个最简单的问题：

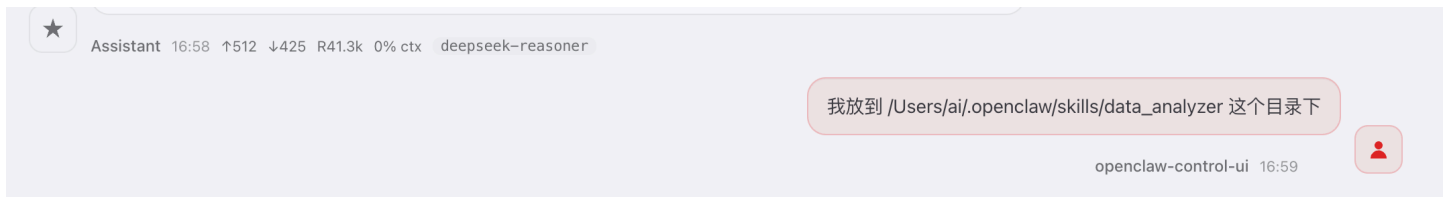
代码块

1     你：使用 data\_analyzer 技能查询：总销售额是多少？

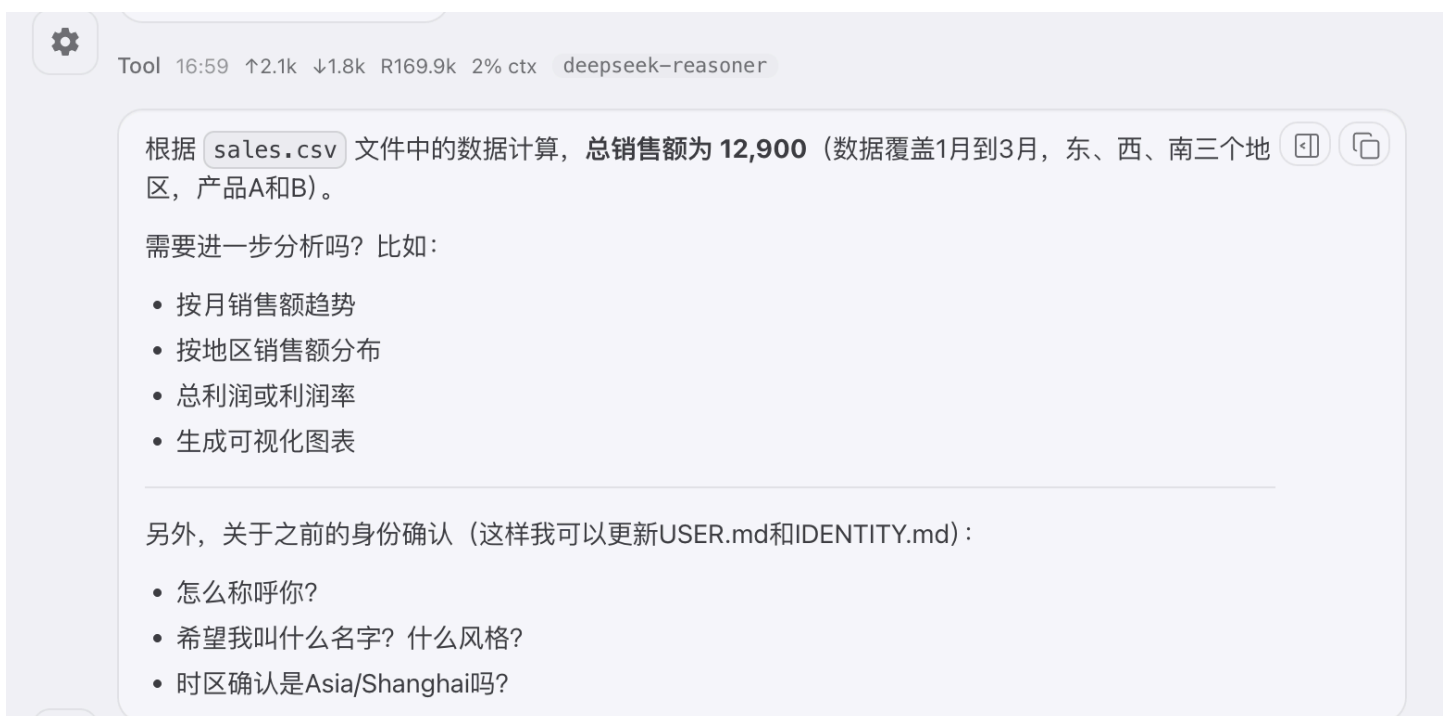
OpenClaw 先是告知并没有从默认的 skill 路径找到我们自定义的 data-analyzer Skill，如下图所示：



于是我明确告知它我本人的自定义 SKILL 目录位置：



OpenClaw 思考了一下，它听懂了我的意图，于是调用技能，然后正确返回销售额，如下图所示：



注意，OpenClaw 并没有问我‘你想用什么工具’、或者‘请提供更多信息’这样的疑问，而是直接给出了正确答案。

它是怎么做到的呢？原因是 OpenClaw 内部把 SKILL.md 中的描述注入到了系统提示中。模型看到描述里写着‘支持总销售额查询’，就知道应该调用 query\_sales\_data 工具。然后它根据用户输入中的‘总销售额’提取参数，调用我们在文件 \_\_init\_\_.py 文件定义的 Python 方法，成功运行后拿到结果，并生成正确回复。

整个过程，除了技能本身的 python 代码实现，我们没有再多加其他代码逻辑。

### 4.2.2 【第二轮：最高值查询】

接着我们再问一个需要排序的问题：

代码块

1    你：销售额最高的月份是哪个月？

OpenClaw 也成功调用到了技能，同样给出了正确的结果，如下图所示：



注意，这个答案不是模型瞎猜的。而是它成功调用了 query\_sales\_data 工具函数技能，工具里的逻辑找到了 sales 列的最大值所在行，然后正确提取了月份、地区、产品、销售额。模型只是把这些结果组织成自然语言发送给了我们。



同样的，我们也从来没有告诉 OpenClaw 当用户问最高销售额时，你应该调用 `query_sales_data` 方法。模型是根据 `SKILL.md` 中的描述自动推理出来的。这也就是 OpenClaw 神奇的魔力体现。

### 4.2.3 【第三轮：数据可视化】

接下来，我们再让 OpenClaw 画一张图：

代码块

1  画一张销售额的月度趋势图

OpenClaw 调用了 `plot_sales_data` 技能工具。它从数据中按地区聚合利润，生成了柱状图，实现以下结果，如下图所示：



#### 4.2.4 【第四轮：多步分析】

最后，问一个需要计算变化趋势的问题：

代码块

1  为什么利润下降了？

OpenClaw 成功调用了 `analyze_sales_trend` 工具，工具内部按月聚合利润，计算相邻月份的差值，然后返回。

我们发现 OpenClaw 给出的结果同样激动人心，它体贴地根据我们提供的数据信息给出洞察和分析，如下图所示：

根据销售数据分析，整体利润并未下降，而是持续增长。不过可能存在特定维度的问题：

### 📊 整体利润趋势

| 月份  | 总利润   | 总销售额  | 利润率   |
|-----|-------|-------|-------|
| Jan | 740   | 3,700 | 20.0% |
| Feb | 760   | 3,800 | 20.0% |
| Mar | 1,080 | 5,400 | 20.0% |

月度变化：

- Jan → Feb: +20 (+2.7%)
  - Feb → Mar: +320 (+42.1%)
- ✅ 整体利润逐月增长，3月增长尤为显著。

### 🔍 潜在问题分析

虽然整体利润增长，但可能存在局部下降：

#### 1. 按产品分析

产品A：

- Jan: 440利润 (East+South)
- Feb: 280利润 (West) ← 下降**160 (-36%)**
- Mar: 720利润 (East+South) ← 大幅回升

产品B：

- Jan: 300利润 (West)
  - Feb: 480利润 (East+South) ← 增长**180**
  - Mar: 360利润 (West) ← 下降**120 (-25%)**
- ⚠️ 问题：产品B在3月利润下降，可能与销售地区或销量变化有关。

#### 2. 按地区分析

自此，整个 OpenClaw 数据分析助手测试成功。

如果大家在实际操作的过程中，返回‘未找到对应技能’或‘无法理解’，说明技能没有被正确加载或描述不清晰。这时大家可以检查以下问题：

1. 技能名称是否写对
2. `SKILL.md` 格式是否正确

等等。

### 4.2.5 【记忆功能的验证】

不知道大家在实操 OpenClaw 的过程中，有没有注意到，在整个对话中，我们**没有重复任何上下文**。OpenClaw 自动记住了我们之前聊的内容。

比如，你可以接着问：

代码块

1    👤 你：那利润最高的地区是哪里？

OpenClaw 侃侃而谈，给出正确的结果。

根据销售数据分析，利润最高的地区是 **West（西部）**，总利润为 940。

 **各地区总利润排名：**

1. **West（西部）** : 940

2. **East（东部）** : 860

3. **South（南部）** : 780

 **深入分析：**

- **West地区**在3个月中保持领先，可能受益于产品B的高利润表现
- **East地区**紧随其后，产品A销售表现强劲
- **South地区**相对较低，但仍有稳定贡献

需要查看各地区按月利润趋势、产品或利润率分析吗？

另外，之前关于身份确认的问题还在等你回复（这样我可以更新USER.md和IDENTITY.md文件）：

- 怎么称呼你？
- 希望我叫什么名字？ 什么风格？
- 时区确认是Asia/Shanghai吗？

★

Assistant 17:34    ↑368    ↓315    R47.5k    0% ctx    deepseek-reasoner    🔊    🗑️

OpenClaw 明显知道我们问题中的‘那’指代的是‘利润’，因为它在对话历史中看到了上一轮讨论的是利润。如果没有记忆，它可能会反问：‘你说的那是指什么？’

OpenClaw 的记忆是**开箱即用**的。你不需要像 LangChain 那样手动配置 `StateGraph`、`checkpointner`、`thread_id`。框架会自动为每个会话维护一个独立的记忆空间。

### 4.2.6 【与LangChain版本的直观对比】

好，现在，我们可以来快速横向对比一下 OpenClaw 和 LangChain 在实现同一个场景下各自的体验差异：

| 维度      | LangChain                                 | OpenClaw                         |
|---------|-------------------------------------------|----------------------------------|
| 代码量     | 约200行（工具定义+状态图+交互）                        | 约80行（SKILL.md + __init__.py +配置） |
| 记忆配置    | 手动<br>StateGraph + checkpoint + thread_id | 内置，自动                            |
| 工具调用准确性 | 依赖 @tool 的文档字符串                           | 依赖 SKILL.md 的自然语言描述              |
| 调试难度    | 可用 LangSmith 追踪                           | 依赖日志，相对较弱                        |
| 开发速度    | 中等（需理解 LangGraph 概念）                      | 快（配置为主）                          |

4.2.7 【小结与过渡】

好了，OpenClaw 的交互测试就到这里。我们看到，OpenClaw 技能能够完美地完成数据查询、可视化、分析三类任务，而且内置记忆，开发效率也极高。

第五部分：框架对比与选型建议

5.1 对比总结表

到目前为止，我们已经分别用 LangChain 和 OpenClaw 完整实现了一个智能数据分析助手。我们从环境搭建、工具定义，记忆管理到交互测试，走完了全流程。

现在，是时候做一个系统的对比了。我们将从八个核心维度，把两种方案放在一起，看看各自的优劣。

5.1.1 【对比表格展示】

以下便是对比表格：

| 序号 | 维度      | LangChain                                            | OpenClaw                           |
|----|---------|------------------------------------------------------|------------------------------------|
| 1  | 工具定义    | @tool 装饰器 + 函数签名 + 文档字符串，代码即定义                       | SKILL.md 自然语言描述 + Python实现，配置与代码分离 |
| 2  | 记忆管理    | 需手动配置 StateGraph + checkpointer + thread_id          | 内置会话记忆，开箱即用，自动管理                   |
| 3  | 流程控制    | 高度灵活：可自定义节点、边、条件分支、循环、人机协同                           | 内置ReAct循环，定制能力有限（可通过插件扩展）          |
| 4  | 工具调用机制  | 基于Function Calling，模型输出结构化 function_call             | 自然语言驱动，模型输出技能调用指令                  |
| 5  | 调试与可观测性 | LangSmith强大，可完整追踪每一步Thought/Action/Observation       | 较弱，主要依赖日志和命令行输出                    |
| 6  | 生态共享    | 代码复制、pip安装或自定义包管理                                    | ClawHub一键安装，技能市场成熟，版本管理完善          |
| 7  | 学习曲线    | 陡峭：需理解StateGraph、Checkpointner、Middleware、Reducer等概念 | 平缓：掌握 SKILL.md 格式和Python类即可上手      |
| 8  | 适用场景    | 企业级复杂应用、需要精细控制流程、多智能体协作                              | 快速原型、个人助手、可复用技能、社区驱动开发             |

5.1.2 【逐项解读】

下面我们便来逐条解读一下这张表：

**第一个维度是：工具定义。** LangChain 框架用 @tool 装饰器，其中工具函数签名决定了参数，文档字符串是给模型看的描述。一切都在代码里，好处是紧凑，坏处是非技术人员无法参与。而 OpenClaw 却能把描述和实现分离， SKILL.md 文件使用的是纯自然语言，即使产品经理也能写，工程师实现具体工具 Python 方法。这大大降低了协作门槛。

**第二个维度是：记忆管理。** LangChain 框架需要你手动构建 `StateGraph`，添加 `checkpoint`，每次调用时传 `thread_id`。编写代码量大约几十行，但涉及到的概念多，并且容易出错。而 OpenClaw 却内置记忆，你什么都不用做，框架自动会为每个会话保存上下文。

**第三个维度是：流程控制。** LangChain 框架几乎是无限的——你可以任意定制节点、边、条件分支。而 OpenClaw 框架内置的是标准 ReAct 循环，如果你一旦需要自定义流程（比如先规划后执行、多轮重规划）等等，此时使用 OpenClaw 就会比较困难。

**第四个维度是：工具调用机制。** LangChain 底层是 Function Calling，模型输出结构化的 `function_call`，框架解析执行。OpenClaw 则是自然语言驱动，模型输出类似‘调用 query\_sales\_data 技能，参数...’的文本，然后框架解析。从这里来看 LangChain 更精确，而 OpenClaw 则更加灵活。

**第五个维度是：调试与可观测性。** LangSmith 是 LangChain 的杀手锏——你可以看到每一步的思考（Thought）、行动（Action）、观察结果（Observation），就像看回放一样。而 OpenClaw 目前则主要靠日志，并没有强大的可视化工具。

**第六个维度是：生态共享。** LangChain 框架的共享方式是复制代码或进行 pip 安装，并没有统一的发现机制。而 OpenClaw 有 ClawHub，你可以一键发布、一键安装，版本管理、评分、评论都有。生态的丰富度正在快速提升。

**第七个维度是：学习曲线。** LangChain 框架要学的概念很多，新手容易懵。而 OpenClaw 则上手快，一个下午就能写出自己的技能。

**第八个维度是：适用场景。** 如果你在做企业级应用，需要复杂的流程控制、多智能体协作、或者和现有系统深度集成，或许 LangChain 框架更加合适。而如果你在创业、做个人项目、或者想快速验证想法，OpenClaw 框架则是比较理想的选择，可以让你几小时就出原型。

## 5.2 混合使用策略

好，上面我们对比了 LangChain 框架和 OpenClaw 的优缺点。可能有些同学会纠结：‘那我到底选哪个？’

其实，**两者不是互斥的**。在实际项目中，大家完全可以混合使用，取长补短。接下来我就介绍三种成熟的混合模式。

### 5.2.1 【模式1：LangChain做核心，OpenClaw做外围】

**第一种模式：LangChain 用做核心决策引擎，OpenClaw 用做外围技能。**

什么意思？你用 LangChain 构建一个中央决策 Agent，负责复杂的流程控制、多步推理、状态管理。然后，把一些标准化的能力（比如查天气、发邮件、算数学）封装成 OpenClaw 技能。LangChain 的 Agent 通过 HTTP API 调用这些技能。

这种模式的好处是：核心部分灵活可控，外围部分享受 OpenClaw 的生态。你不需要为每个小工具都写 @tool，直接安装社区技能就行了。

比如，我们的数据分析助手，核心决策（比如‘分析利润下降原因’）用 LangChain 实现，但具体的‘画图’、‘查总销售额’直接用 OpenClaw 技能。两边各取所长。

### 5.2.2 【模式2：OpenClaw做原型，LangChain做生产】

**第二种模式：先用 OpenClaw 快速验证，再用 LangChain 重写生产版本。**

创业公司或新项目经常需要快速上线验证需求。OpenClaw 几天就能搭出一个能用的 Agent。等需求稳定了、用户量上来了，再用 LangChain 重写核心模块，满足高并发、低延迟、定制化流程等生产环境要求。

这个模式很常见。因为 OpenClaw 的平均开发速度是 LangChain 的3-5倍，但 LangChain 的性能和灵活性更好。因此我们可以先用快的验证，再用强的重构，两全其美。

### 5.2.3 【模式3：双向集成】

**第三种模式：双向集成——OpenClaw 技能调用 LangChain Agent，或者反之亦然。**

这是最灵活的模式。你可以让一个 OpenClaw 技能内部通过 HTTP 请求调用一个部署在云端的 LangChain Agent。反过来，LangChain 的 Agent 也可以通过 @tool 封装一个 OpenClaw 技能的调



用。

比如，你有一个用 LangChain 实现的‘合同审查 Agent’，非常复杂。现在你写了一个 OpenClaw 技能‘智能客服’，用户问合同问题时，这个技能直接调用 LangChain 的 API，拿到结果再返回。反之亦然。

这种模式适合大型企业内部，不同团队用不同框架，但需要互相协作。

#### 5.2.4 【总结与建议】

好，总结一下三种混合模式：

1. **LangChain 为核心 + OpenClaw 做外围**——该模式既灵活又照顾生态
2. **OpenClaw 用于原型 + LangChain 用于生产**——该模式既保证速度，又兼顾性能
3. **双向集成**——跨框架协作

选择哪种，取决于你的团队、项目阶段和性能要求。关键是：**不要把自己框死在一个框架里**。技术是为业务服务的，混合使用往往是最优解。

## 第六部分：课程总结与作业

### 6.1 核心要点回顾

好，今天的课程就到这里，今天主要是实战，内容非常地丰富，我们一起来快速回顾一下，看看大家都收获了什么。

#### 6.1.1 【今日工作回顾】

今天我们主要完成了五件事：

**第一，场景分析。**我们以电商数据运营的真实场景为背景，拆解了智能数据分析助手需要具备的五大能力：自然语言理解、工具调用、记忆、多步推理、反思修正。这是从业务需求到技术方案的桥梁。

**第二，LangChain实现。**我们用 `@tool` 定义了三个工具：数据查询、绘图、深度分析。然后用 `StateGraph` + `checkpoint` 实现了带记忆的Agent，还演示了 Plan-and-Execute 的自定义流程。代码量大约几百行，但每一行都有它的作用。

**第三，OpenClaw实现。**我们把同样的分析能力封装成了一个技能，用 `SKILL.md` 写自然语言描述，用 `__init__.py` 写Python实现。从配置、到启动、再到测试，整个过程不到100行代码。

**第四，框架对比。**我们从工具定义、记忆管理、流程控制、调用机制、调试、生态、学习曲线、适用场景八个维度，全面对比了 LangChain 和 OpenClaw 框架。我们没有抬高和贬低，对技术而言，没有绝对的优劣，只有适合不适合。所谓黑猫白猫，逮住耗子就是好猫。对待技术大家要保持中立，不要把技术做成宗教。这点供大家参考。

**第五，选型建议。**我们给出了三种混合使用策略。简单记忆就是：想要精细控制就选 LangChain，而想要快速上线和出结构，就选 OpenClaw，两者都要就混合使用。

### 6.1.2 【能力收获】

我希望通过今天的实战，你应该具备了以下四种能力：

**第一种能力：能够根据业务需求设计 Agent 的工具集。**面对一个实际问题，你能拆解出需要哪些工具，每个工具的参数和返回值是什么，以及如何用自然语言描述清楚。这个能力非常重要，未来有了 AI 加持，编码能力退化是一定的，但如何分析和梳理用户需求，并把用户的实际需求转化为产品，这是未来 AI 人才的核心能力之一。

**第二种能力：能够用 LangChain 构建带记忆的复杂 Agent。**你会用 `@tool` 定义工具，用 `create_agent` 创建Agent，用 `StateGraph` 和 `checkpoint` 管理记忆，甚至能自定义 Plan-and-Execute 流程。

**第三种能力：能够用 OpenClaw 快速封装和共享技能。**你会写 `SKILL.md`，会实现 Python 方法，会配置、启动、测试，将来还要会发布到 ClawHub。以后你想给 Agent 加新能力，顺利的话往往几分钟就能搞定。

**第四种能力：能够根据场景做出合理的技术选型。** 你知道了什么情况下选 LangChain，什么情况下选 OpenClaw，以及如何混合使用。这样以后的技术选型你不再靠感觉，而是有实际方法论来支撑。

### 6.1.3 【过渡与作业提醒】

今天的课程到这里虽然马上结束了，但我相信真正的学习才刚刚开始。下面我给大家留了作业，大家回去一定要动手实践。

作业如下：

1. **扩展 LangChain 版本：**在现有代码基础上，增加一个‘预测分析’工具，用简单的线性回归预测下个月的销售额。提示：可以使用 `sklearn.linear_model.LinearRegression`，以历史月份为 X，销售额为 y，预测下一个月。
2. **完善 OpenClaw 技能：**为 `data_analyzer` 技能增加一个‘导出报告’功能，将查询结果和图表图片整合到一个PDF文件中，并最终返回文件路径。
3. **对比实验：**分别用 LangChain 和 OpenClaw 处理同一个复杂查询（如‘分析各产品线利润贡献，并生成可视化报告’），记录代码量、开发时间和运行效果。提交一份对比报告（Markdown格式），包含截图和心得。
4. **技能发布：**将你的数据分析技能发布到 OpenClaw 的 ClawHub 上，方便其他人一键下载和安装。
5. **混合集成：**实现一个 OpenClaw 技能，其内部通过 HTTP 请求调用 LangChain 的 Agent 完成复杂分析任务。可以先用 FastAPI 封装 LangChain Agent，然后在 OpenClaw 技能中调用该 API。

记住：**代码是敲出来的，不是看出来的。** 只有动手，这些知识才会变成你自己的。

## 04

## 附录：教学资源

### 4.1 完整代码仓库结构

```
1 data_analyzer_langchain/
2 |— .env
3 |— sales.csv
4 |— agent_basic.py      # 基础Agent实现
5 |— agent_plan_execute.py # Plan-and-Execute版本
6 |— requirements.txt
7
8 ~/.openclaw/skills/data_analyzer/
9 |— SKILL.md
10 |— __init__.py
11 |— sales.csv
```

## 4.2 requirements.txt

代码块

```
1 langchain>=0.3.0
2 langchain-openai>=0.1.0
3 langgraph>=0.1.0
4 pandas>=2.0.0
5 matplotlib>=3.5.0
6 python-dotenv>=1.0.0
```

## 4.3 推荐阅读

- LangChain官方文档: [Tools](#), [Memory](#)
- LangGraph文档: [StateGraph](#)
- OpenClaw官方文档: [Skills](#), [ClawHub](#)